23CSE455 Design Patterns

Case Study Report

Team: B16

Case study: Testing frameworks (Java): *Design Pattern Analysis and Evolution in the JUnit 6 Testing Framework*

Application: JUnit 6.1.3, the application named in the assigned case study

Date: 2026-10-10

Submission repository: 23CSE455_DP_CaseStudy, folder `B16/`

Students

Roll number	Student name	Batch
AM.SC.U4CSE23319	Gautham Mohanraj	CSE D

1 Application and architecture

Original source: `https://github.com/junit-team/junit-framework`, tag `r6.1.3`, baseline commit `f59f60d2cebdf2224235d81f781b1f310cbc8138` ("Release 6.1.3"), accessed 2026-10-10, Eclipse Public License 2.0. An unchanged copy of the inspected code is in `data/code/`.

Environment: Java library built with Gradle 9.5.1 (wrapper in the repository); JUnit requires Java 17 or higher at runtime. Built and tested with OpenJDK 25.0.4.1 on a Mac with Apple M3 (Darwin arm64). No database. Part 3 used Python 3.14.2, Node.js 26.11.1 and Apple clang 21.0.0 (C++20). Metrics: lizard 1.24.1. Diagrams: PlantUML.

Scope: Production code (`src/main/java`) of four modules: `junit-platform-engine`, `junit-platform-launcher`, `junit-jupiter-api`, `junit-jupiter-engine`. They cover the four areas of the case study: (A) test discovery and descriptors, (B) engines and the execution lifecycle, (C) execution listeners and reporting, (D) Jupiter extensions and resource management. Excluded: `junit-platform-commons` (utilities used by all modules), console, suite, reporting, testkit, Vintage and params modules, and all test code, because none of them implements one of the four areas; tests are only used to run and verify the changes.

Architecture: Figure 1 shows one test run. A client (IDE, build tool or console) builds a `LauncherDiscoveryRequest` and calls the `Launcher` created by `LauncherFactory`. `EngineDiscoveryOrchestrator` asks every `TestEngine` to discover a tree of `TestDescriptor`s, applies post-discovery filters, prunes empty containers, and `DefaultLauncher` turns the result into a `TestPlan`. `EngineExecutionOrchestrator` then lets each engine execute its tree: in Jupiter, `HierarchicalTestEngine` creates a `NodeTestTask` per descriptor, which runs the Jupiter lifecycle (extensions, `@BeforeEach`, the test, cleanup) and reports every event to an `EngineExecutionListener`. `ExecutionListenerAdapter` converts those engine events into `TestExecutionListener` events, which `CompositeTestExecutionListener` broadcasts to all registered listeners (IDE, summary, reports).

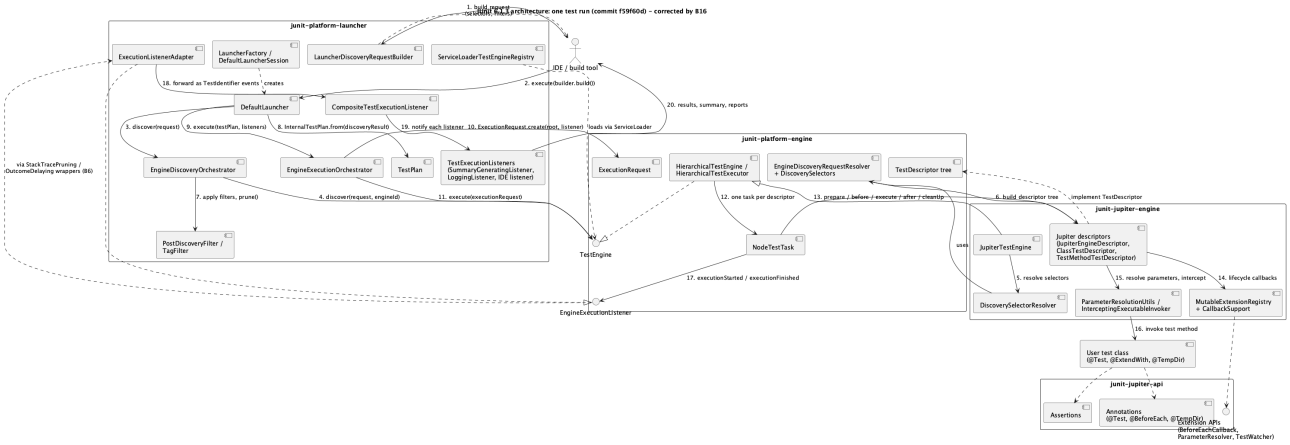


Figure 1: Components and numbered flow of one test run (`Part1/architecture/architecture.puml`; the LLM's original version and the two corrections are in the same folder).

Run instructions: `README.md` (team folder). The JUnit fork is built and tested from its root with `./gradlew--no-configuration-cache--no-build-cache:platform-tests:test--rerun--tests"org.junit.platform.launcher.*"` (and `:jupiter-tests:test`). Actual results at the baseline: 382/382 launcher tests, 2452 platform tests and 3381 Jupiter tests passed (0 failures; 6 and 165 skipped by design). Logs: `Part2/changes/baseline_launcher_tests.log`, `Part2/changes/baseline_all_platform_tests.log`, `Part2/changes/Change2/before_all_jupiter_tests.log`.

2 Part 1 Pattern discovery

Method: For each area I first asked the LLM (Claude Opus 5.5) to list the files and explain the runtime flow, checked that answer in the source, and only then asked for pattern candidates with roles, files and line numbers (12 prompts, `Part1/llm/`). Every claimed line was checked automatically (`Part1/evidence/check_lines.py`) and every role by reading the code; 102 claims are recorded in `Part1/evidence/verification.csv`. Two counting rules were fixed before counting: (1) *Composite rule*: a class is a Composite only if it combines children of its own interface *and* merges their answers into one result; (2) two views of the same classes (A2/A3, B1/B2) are counted as separate instances with an overlap note, and a class found in two areas is counted once (B6–B8 are marked "also Area C"). Line ranges in `data/patterns.csv` are computed from the source by `data/build_part1_csv.py`.

2.1 Totals at the baseline

Confirmed pattern types: 10 (Strategy, Adapter, Observer, Decorator, Template Method, Composite, Command, Null Object, Chain of Responsibility, Factory Method)

Confirmed pattern instances: 35

Concrete participant classes: 76

Unique participant symbols: 152 (177 role mappings; symbols shared by two instances counted once)

Rejected and uncertain candidates: 25 rejected and 8 uncertain, observed in the same four modules at commit `f59f60d` (Table 37).

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Strategy	A3 Post-Discovery Filter Policies; B3 Test Executor Services; B4 Parallelism Configuration Strategy; C3 Listener Notification Order; C4 Pluggable Log Sink; D1 Temp Directory Creation; D2 Temp Directory Deletion; D3 Parameter Resolution; D4 Conditional Execution; D5 Store Close Action	10	21
Adapter	A7 Orderer Descriptor Wrappers; B7 Engine-to-Launcher Listener Adapter; C5 Test Reporter Adapter; D6 Extension Store Adapter; D7 Lifecycle Method Adapters	5	5
Observer	A6 Discovery Event Notification; B8 Execution Event Notification; C1 Test Execution Listeners; C2 Launcher Session Notification; D8 Test Outcome Watchers	5	11
Decorator	B5 Intercepting Launcher Wrappers; B6 Engine Listener Wrappers; C6 Captured Output Stream; D10 Timeout Invocation Wrappers	4	9
Template Method	A4 Jupiter Descriptor Skeletons; A5 Child Ordering Skeleton; B1 Hierarchical Engine Execution Skeleton	3	6
Command	B9 Node Execution Tasks; D11 Packaged Reflective Calls	2	3
Composite	A1 Test Descriptor Tree; A2 Composed Discovery Filter	2	15
Null Object	B10 No-op Node; C7 No-op Listeners	2	3
Chain of Responsibility	D9 Invocation Interceptor Chain	1	1
Factory Method	B2 Engine Collaborator Factory Methods	1	2
Total	10 pattern types	**35**	**76**

2.3 One record for each pattern instance

Each record gives the pattern, the purpose (as evidence) and every participant with its file and line range. Paths are shortened to module/.../file; the full paths are in `data/patterns.csv`. All instances are JUnit framework code. Diagrams are in Appendix A.

Area A: discovery and test descriptors

A1 Test Descriptor Tree **Pattern:** Composite (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 12.

Role	Class / interface / function	File and lines
Component	`org.junit.platform.engine.TestDescriptor`	`junit-platform-engine/.../TestDescriptor.java:37-427`
Composite (base)	`org.junit.platform.engine.support.descriptor.AbstractTestDescriptor`	`junit-platform-engine/.../AbstractTestDescriptor.java:42-248`
Composite	`org.junit.platform.engine.support.descriptor.EngineDescriptor`	`junit-platform-engine/.../EngineDescriptor.java:25-53`
Composite	`org.junit.jupiter.engine.descriptor.JupiterEngineDescriptor`	`junit-jupiter-engine/.../JupiterEngineDescriptor.java:30-70`

Role	Class / interface / function	File and lines
Composite	`org.junit.jupiter.engine.descriptor.ClassTestDescriptor`	`junit-jupiter-engine/.../ClassTestDescriptor.java:47-106`
Composite	`org.junit.jupiter.engine.descriptor.NestedClassTestDescriptor`	`junit-jupiter-engine/.../NestedClassTestDescriptor.java:49-121`
Composite	`org.junit.jupiter.engine.descriptor.ClassTemplateTestDescriptor`	`junit-jupiter-engine/.../ClassTemplateTestDescriptor.java:51-277`
Composite	`org.junit.jupiter.engine.descriptor.ClassTemplateInvocationTestDescriptor`	`junit-jupiter-engine/.../ClassTemplateInvocationTestDescriptor.java:48-202`
Composite	`org.junit.jupiter.engine.descriptor.TestTemplateTestDescriptor`	`junit-jupiter-engine/.../TestTemplateTestDescriptor.java:41-167`
Composite	`org.junit.jupiter.engine.descriptor.TestFactoryTestDescriptor`	`junit-jupiter-engine/.../TestFactoryTestDescriptor.java:56-223`
Composite	`org.junit.jupiter.engine.descriptor.DynamicContainerTestDescriptor`	`junit-jupiter-engine/.../DynamicContainerTestDescriptor.java:34-88`
Leaf	`org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor`	`junit-jupiter-engine/.../TestMethodTestDescriptor.java:71-293`
Leaf	`org.junit.jupiter.engine.descriptor.TestTemplateInvocationTestDescriptor`	`junit-jupiter-engine/.../TestTemplateInvocationTestDescriptor.java:42-104`
Leaf	`org.junit.jupiter.engine.descriptor.DynamicTestTestDescriptor`	`junit-jupiter-engine/.../DynamicTestTestDescriptor.java:37-100`
Client	`org.junit.platform.launcher.core.EngineDiscoveryOrchestrator#applyPostDiscoveryFilters`	`junit-platform-launcher/.../EngineDiscoveryOrchestrator.java:225-238`
Client	`org.junit.platform.engine.support.hierarchical.NodeTestTask#executeRecursively`	`junit-platform-engine/.../NodeTestTask.java:157-189`

Evidence: Engines build one tree of TestDescriptor nodes (engine > class > nested class > method > dynamic test); the launcher filters, prunes, visits and executes it without knowing node kinds. Evidence: TestDescriptor.accept visits itself then calls child.accept(visitor); AbstractTestDescriptor holds the children set and findByUniqueId recurses; NodeTestTask creates one child task per child. Transparent variant: leaf vs container is decided by getType().

Key code: `this.getChildren().forEach(child -> child.accept(visitor));` (TestDescriptor.java:348)

Diagram: `Part1/uml/png/A1.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A2 Composed Discovery Filter Pattern: Composite (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Component	`org.junit.platform.engine.Filter`	`junit-platform-engine/.../Filter.java:41-119`
Composite	`org.junit.platform.engine.CompositeFilter`	`junit-platform-engine/.../CompositeFilter.java:29-89`
Leaf	`org.junit.platform.launcher.IncludeMethodFilter`	`junit-platform-launcher/../IncludeMethodFilter.java:31-58`
Leaf	`org.junit.platform.launcher.ExcludeMethodFilter`	`junit-platform-launcher/../ExcludeMethodFilter.java:31-58`
Leaf (lambda factory)	`org.junit.platform.launcher.TagFilter`	`junit-platform-launcher/../TagFilter.java:48-184`
Client	`org.junit.platform.launcher.core.EngineDiscoveryOrchestrator#applyPostDiscoveryFilters`	`junit-platform-launcher/../EngineDiscoveryOrchestrator.java:225-238`

Evidence: All post-discovery filters are merged into one Filter by Filter.composeFilters; CompositeFilter implements Filter, holds a list of Filters and combines their FilterResults into one answer (first exclusion wins). Overlaps A3 (same leaf classes seen as strategies).

Key code: `.map(filter -> filter.apply(element))` (CompositeFilter.java:62)

Diagram: `Part1/uml/png/A2.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A3 Post-Discovery Filter Policies **Pattern:** Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Strategy	`org.junit.platform.launcher.PostDiscoveryFilter`	`junit-platform-launcher/../PostDiscoveryFilter.java:35-36`
ConcreteStrategy	`org.junit.platform.launcher.IncludeMethodFilter`	`junit-platform-launcher/../IncludeMethodFilter.java:31-58`
ConcreteStrategy	`org.junit.platform.launcher.ExcludeMethodFilter`	`junit-platform-launcher/../ExcludeMethodFilter.java:31-58`
ConcreteStrategy (lambdas)	`org.junit.platform.launcher.TagFilter`	`junit-platform-launcher/../TagFilter.java:48-184`
Context	`org.junit.platform.launcher.core.EngineDiscoveryOrchestrator#applyPostDiscoveryFilters`	`junit-platform-launcher/../EngineDiscoveryOrchestrator.java:225-238`

Evidence: Selection rules (tag expressions, method-name patterns, custom filters) are interchangeable algorithms behind PostDiscoveryFilter; EngineDiscoveryOrchestrator only calls apply(). Overlaps A2.

Key code: `FilterResult filterResult = postDiscoveryFilter.apply(descriptor);` (EngineDiscoveryOrchestrator.java:230)

Diagram: `Part1/uml/png/A3.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed

(reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A4 Jupiter Descriptor Skeletons **Pattern:** Template Method (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
AbstractClass	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:64-283`
TemplateMethod	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor#getExecutionMode`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:163-167`
TemplateMethod	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor#getLegacyReportingName`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:85-88`
TemplateMethod	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor#copyIncludingDescendants`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:256-263`
PrimitiveOperation	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor#withUniqueId`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:268-268`
ConcreteClass (hook)	`org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor#getExplicitExecutionMode`	`junit-jupiter-engine/.../ClassBasedTestDescriptor.java:167-169`
ConcreteClass (hook)	`org.junit.jupiter.engine.descriptor.MethodBasedTestDescriptor#getExplicitExecutionMode`	`junit-jupiter-engine/.../MethodBasedTestDescriptor.java:147-149`
ConcreteClass (primitive)	`org.junit.jupiter.engine.descriptor.ClassTestDescriptor#withUniqueId`	`junit-jupiter-engine/.../ClassTestDescriptor.java:63-66`

Evidence: JupiterTestDescriptor fixes algorithms in final methods (getExecutionMode, getLegacyReportingName) and in copyIncludingDescendants, which calls the abstract primitive withUniqueId; subclasses override the hooks (getExplicitExecutionMode, getLegacyReportingBaseName, withUniqueId).

Key code: `public final ExecutionMode getExecutionMode() {` (JupiterTestDescriptor.java:163; see JupiterTestDescriptor.java:163, 268)

Diagram: `Part1/uml/png/A4.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A5 Child Ordering Skeleton **Pattern:** Template Method (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
AbstractClass	`org.junit.jupiter.engine.discovery.AbstractOrderingVisitor`	`junit-jupiter-engine/.../AbstractOrderingVisitor.java:39-222`

Role	Class / interface / function	File and lines
TemplateMethod	`org.junit.jupiter.engine.discovery.AbstractOrderingVisitor#orderChildrenTestDescriptors`	`junit-jupiter-engine/.../AbstractOrderingVisitor.java:72-119`
PrimitiveOperation	`org.junit.jupiter.engine.discovery.AbstractOrderingVisitor#shouldNonMatchingDescriptorsComeBeforeOrderedOnes`	`junit-jupiter-engine/.../AbstractOrderingVisitor.java:126-126`
ConcreteClass	`org.junit.jupiter.engine.discovery.ClassOrderingVisitor`	`junit-jupiter-engine/.../ClassOrderingVisitor.java:38-165`
ConcreteClass	`org.junit.jupiter.engine.discovery.MethodOrderingVisitor`	`junit-jupiter-engine/.../MethodOrderingVisitor.java:41-174`

Evidence: AbstractOrderingVisitor.orderChildrenTestDescriptors runs a fixed ordering algorithm (select, wrap, sort, unwrap, merge) and asks the abstract hook shouldNonMatchingDescriptorsComeBeforeOrderedOnes, which ClassOrderingVisitor and MethodOrderingVisitor answer differently.

Key code: `if (shouldNonMatchingDescriptorsComeBeforeOrderedOnes()) {` (AbstractOrderingVisitor.java:110; see AbstractOrderingVisitor.java:110, 126)

Diagram: `Part1/uml/png/A5.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A6 Discovery Event Notification Pattern: Observer (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Observer	`org.junit.platform.launcher.LauncherDiscoveryListener`	`junit-platform-launcher/../LauncherDiscoveryListener.java:42-91`
Observer (base)	`org.junit.platform.engine.EngineDiscoveryListener`	`junit-platform-engine/.../EngineDiscoveryListener.java:31-67`
ConcreteObserver	`org.junit.platform.launcher.core.DiscoveryIssueCollector`	`junit-platform-launcher/../DiscoveryIssueCollector.java:25-65`
ConcreteObserver	`org.junit.platform.launcher.core.DiscoveryIssueReportingDiscoveryListener`	`junit-platform-launcher/.../DiscoveryIssueReportingDiscoveryListener.java:49-131`
Subject (attach)	`org.junit.platform.launcher.core.DefaultLauncher#registerLauncherDiscoveryListeners`	`junit-platform-launcher/.../DefaultLauncher.java:73-77`
Subject (notify)	`org.junit.platform.launcher.core.EngineDiscoveryOrchestrator#discoverEngineRoot`	`junit-platform-launcher/../EngineDiscoveryOrchestrator.java:189-214`
Subject (notify, in engine)	`org.junit.platform.engine.support.discovery.EngineDiscoveryRequestResolution`	`junit-platform-engine/.../EngineDiscoveryRequestResolution.java:60-263`

Role	Class / interface / function	File and lines
Multicaster	`org.junit.platform.launcher.listeners.discovery.CompositeLauncherDiscoveryListener`	`junit-platform-launcher/../CompositeLauncherDiscoveryListener.java:29-67`

Evidence: Discovery progress (launcher/engine discovery started and finished, selector processed, issues) is pushed to registered LauncherDiscoveryListeners that the orchestrator only knows by interface.

Key code: `listener.engineDiscoveryStarted(uniqueEngineId);` (EngineDiscoveryOrchestrator.java:194; see EngineDiscoveryOrchestrator.java:194-197)

Diagram: `Part1/uml/png/A6.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

A7 Orderer Descriptor Wrappers **Pattern:** Adapter (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Target	`org.junit.jupiter.api.ClassDescriptor`	`junit-jupiter-api/.../ClassDescriptor.java:28-87`
Target	`org.junit.jupiter.api.MethodDescriptor`	`junit-jupiter-api/.../MethodDescriptor.java:29-90`
Adapter (base)	`org.junit.jupiter.engine.discovery.AbstractAnnotatedDescriptorWrapper`	`junit-jupiter-engine/.../AbstractAnnotatedDescriptorWrapper.java:28-65`
Adapter	`org.junit.jupiter.engine.discovery.DefaultClassDescriptor`	`junit-jupiter-engine/.../DefaultClassDescriptor.java:23-39`
Adapter	`org.junit.jupiter.engine.discovery.DefaultMethodDescriptor`	`junit-jupiter-engine/.../DefaultMethodDescriptor.java:25-41`
Adaptee	`org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor`	`junit-jupiter-engine/.../ClassBasedTestDescriptor.java:89-601`
Adaptee	`org.junit.jupiter.engine.descriptor.MethodBasedTestDescriptor`	`junit-jupiter-engine/.../MethodBasedTestDescriptor.java:56-217`
Client	`org.junit.jupiter.engine.discovery.ClassOrderingVisitor`	`junit-jupiter-engine/.../ClassOrderingVisitor.java:38-165`

Evidence: User ClassOrderer/MethodOrderer code expects the public ClassDescriptor/MethodDescriptor API; the wrappers expose internal descriptors through it so internal classes never leak to user code.

Key code: `class DefaultClassDescriptor extends AbstractAnnotatedDescriptorWrapper<Class<?>> implements ClassDescriptor {` (DefaultClassDescriptor.java:23; see DefaultClassDescriptor.java:23-26)

Diagram: `Part1/uml/png/A7.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

Area B: engines and execution lifecycle

B1 Hierarchical Engine Execution Skeleton **Pattern:** Template Method (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
AbstractClass	`org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine`	`junit-platform-engine/.../HierarchicalTestEngine.java:31-119`
TemplateMethod	`org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine#execute`	`junit-platform-engine/.../HierarchicalTestEngine.java:53-63`
PrimitiveOperation	`org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine#createExecutionContext`	`junit-platform-engine/.../HierarchicalTestEngine.java:117-117`
ConcreteClass	`org.junit.jupiter.engine.JupiterTestEngine`	`junit-jupiter-engine/.../JupiterTestEngine.java:44-108`

Evidence: final HierarchicalTestEngine.execute fixes the run order (executor service, context, collector factory, HierarchicalTestExecutor); JupiterTestEngine only supplies the steps. Overlaps B2.

Key code: `public final void execute(ExecutionRequest request) {` (HierarchicalTestEngine.java:53; see HierarchicalTestEngine.java:53-59, 117)

Diagram: `Part1/uml/png/B1.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B2 Engine Collaborator Factory Methods **Pattern:** Factory Method (Creational, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Creator (factory method)	`org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine#createExecutorService`	`junit-platform-engine/.../HierarchicalTestEngine.java:83-85`
ConcreteCreator	`org.junit.jupiter.engine.JupiterTestEngine#createExecutorService`	`junit-jupiter-engine/.../JupiterTestEngine.java:80-87`
ConcreteCreator	`org.junit.jupiter.engine.JupiterTestEngine#createExecutionContext`	`junit-jupiter-engine/.../JupiterTestEngine.java:90-93`
ConcreteCreator	`org.junit.jupiter.engine.JupiterTestEngine#createThrowableCollectorFactory`	`junit-jupiter-engine/.../JupiterTestEngine.java:99-101`
ConcreteProduct	`org.junit.jupiter.engine.execution.JupiterEngineExecutionContext`	`junit-jupiter-engine/.../JupiterEngineExecutionContext.java:30-209`

Evidence: The template in B1 asks overridable create* methods which objects to use; JupiterTestEngine overrides them to choose a parallel/same-thread executor, its JupiterEngineExecutionContext and its throwable-collector factory. Overlaps B1.

Key code: `protected HierarchicalTestExecutorService createExecutorService(ExecutionRequest request) {` (JupiterTestEngine.java:80; see JupiterTestEngine.java:80-100)

Diagram: Part1/uml/png/B2.png (source in Part1/uml/, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in Part1/evidence/verification.csv, decisions in Part1/notes.md.

B3 Test Executor Services Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Strategy	org.junit.platform.engine.support.hierarchical.HierarchicalTestExecutorService	junit-platform-engine/.../HierarchicalTestExecutorService.java:34-118
ConcreteStrategy	org.junit.platform.engine.support.hierarchical.SameThreadHierarchicalTestExecutorService	junit-platform-engine/.../SameThreadHierarchicalTestExecutorService.java:29-50
ConcreteStrategy	org.junit.platform.engine.support.hierarchical.ForkJoinPoolHierarchicalTestExecutorService	junit-platform-engine/.../ForkJoinPoolHierarchicalTestExecutorService.java:51-331
ConcreteStrategy	org.junit.platform.engine.support.hierarchical.WorkerThreadPoolHierarchicalTestExecutorService	junit-platform-engine/.../WorkerThreadPoolHierarchicalTestExecutorService.java:70-901
Context	org.junit.platform.engine.support.hierarchical.HierarchicalTestExecutor	junit-platform-engine/.../HierarchicalTestExecutor.java:36-71
Context	org.junit.platform.engine.support.hierarchical.NodeTestTask#executeRecursively	junit-platform-engine/.../NodeTestTask.java:157-189

Evidence: Sequential vs parallel scheduling is swappable behind HierarchicalTestExecutorService; NodeTestTask and HierarchicalTestExecutor only call submit/invokeAll.

Key code: `taskContext.executorService().invokeAll(children);` (NodeTestTask.java:180)

Diagram: Part1/uml/png/B3.png (source in Part1/uml/, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in Part1/evidence/verification.csv, decisions in Part1/notes.md.

B4 Parallelism Configuration Strategy Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Strategy	org.junit.platform.engine.support.hierarchical.ParallelExecutionConfigurationStrategy	junit-platform-engine/.../ParallelExecutionConfigurationStrategy.java:25-33
ConcreteStrategy (enum FIXED/ DYNAMIC/ CUSTOM)	org.junit.platform.engine.support.hierarchical.DefaultParallelExecutionConfigurationStrategy	junit-platform-engine/.../DefaultParallelExecutionConfigurationStrategy.java:33-228

Role	Class / interface / function	File and lines
Selection	`org.junit.platform.engine.support.hierarchical.DefaultParallelExecutionConfigurationStrategy#getStrategy`	`junit-platform-engine/.../DefaultParallelExecutionConfigurationStrategy.java:223-226`
Context	`org.junit.platform.engine.support.hierarchical.ParallelHierarchicalTestExecutorServiceFactory`	`junit-platform-engine/.../ParallelHierarchicalTestExecutorServiceFactory.java:33-123`

Evidence: How parallelism is computed (FIXED, DYNAMIC from CPU count, CUSTOM user class) is chosen by configuration; each enum constant implements createConfiguration.

Key code: `return getStrategy(configurationParameters).createConfiguration(configurationParameters);` (DefaultParallelExecutionConfigurationStrategy.java:220; see DefaultParallelExecutionConfigurationStrategy.java:220-225)

Diagram: `Part1/uml/png/B4.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B5 Intercepting Launcher Wrappers **Pattern:** Decorator (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Component	`org.junit.platform.launcher.Launcher`	`junit-platform-launcher/../Launcher.java:53-164`
ConcreteComponent	`org.junit.platform.launcher.core.DefaultLauncher`	`junit-platform-launcher/.../DefaultLauncher.java:43-145`
Decorator	`org.junit.platform.launcher.core.DelegatingLauncher`	`junit-platform-launcher/../DelegatingLauncher.java:24-73`
ConcreteDecorator	`org.junit.platform.launcher.core.InterceptingLauncher`	`junit-platform-launcher/../InterceptingLauncher.java:25-70`
Assembler	`org.junit.platform.launcher.core.DefaultLauncherSession`	`junit-platform-launcher/../DefaultLauncherSession.java:35-159`

Evidence: The session's launcher is DelegatingLauncher > InterceptingLauncher > DefaultLauncher; each wrapper implements Launcher and forwards, InterceptingLauncher adds LauncherInterceptor around every call.

Key code: `launcher = new InterceptingLauncher(interceptor.intercept(() -> launcherFactory.apply(this.store)),` (DefaultLauncherSession.java:67; see DefaultLauncherSession.java:67-70)

Diagram: `Part1/uml/png/B5.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B6 Engine Listener Wrappers **Pattern:** Decorator (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Component	`org.junit.platform.engine.EngineExecutionListener`	`junit-platform-engine/.../EngineExecutionListener.java:34-164`
Decorator	`org.junit.platform.launcher.core.DelegatingEngineExecutionListener`	`junit-platform-launcher/../DelegatingEngineExecutionListener.java:22-59`
ConcreteDecorator	`org.junit.platform.launcher.core.StackTracePruningEngineExecutionListener`	`junit-platform-launcher/../StackTracePruningEngineExecutionListener.java:31-70`
ConcreteDecorator	`org.junit.platform.launcher.core.OutcomeDelayingEngineExecutionListener`	`junit-platform-launcher/../OutcomeDelayingEngineExecutionListener.java:28-101`
ConcreteDecorator	`org.junit.platform.launcher.core.MemoryCleanupListener`	`junit-platform-launcher/../MemoryCleanupListener.java:30-93`
Assembler	`org.junit.platform.launcher.core.EngineExecutionOrchestrator#executeEngine`	`junit-platform-launcher/../EngineExecutionOrchestrator.java:248-275`

Evidence: EngineExecutionOrchestrator stacks independent wrappers (stack-trace pruning, outcome delaying, memory cleanup) on the engine listener; each forwards and adds one behaviour. Also part of Area C.

Key code: `listener = new MemoryCleanupListener(listener, testPlan);` (EngineExecutionOrchestrator.java:149; see EngineExecutionOrchestrator.java:149, 198, 253)

Diagram: `Part1/uml/png/B6.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B7 Engine-to-Launcher Listener Adapter **Pattern:** Adapter (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Target	`org.junit.platform.engine.EngineExecutionListener`	`junit-platform-engine/.../EngineExecutionListener.java:34-164`
Adapter	`org.junit.platform.launcher.core.ExecutionListenerAdapter`	`junit-platform-launcher/../ExecutionListenerAdapter.java:28-74`
Adaptee	`org.junit.platform.launcher.TestExecutionListener`	`junit-platform-launcher/../TestExecutionListener.java:76-203`
Client (wiring)	`org.junit.platform.launcher.core.EngineExecutionOrchestrator#buildEngineExecutionListener`	`junit-platform-launcher/../EngineExecutionOrchestrator.java:141-152`

Evidence: Engines report with mutable TestDescriptors; ExecutionListenerAdapter implements EngineExecutionListener and forwards to a TestExecutionListener with immutable TestIdentifiers looked

up in the TestPlan. Also part of Area C.

Key code: `this.testExecutionListener.executionStarted(getTestIdentifier(testDescriptor));` (ExecutionListenerAdapter.java:47)

Diagram: `Part1/uml/png/B7.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B8 Execution Event Notification **Pattern:** Observer (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Observer	`org.junit.platform.engine.EngineExecutionListener`	`junit-platform-engine/.../EngineExecutionListener.java:34-164`
ConcreteObserver	`org.junit.platform.launcher.core.OutcomeDelayingEngineExecutionListener`	`junit-platform-launcher/.../OutcomeDelayingEngineExecutionListener.java:28-101`
ConcreteObserver	`org.junit.platform.launcher.core.ExecutionListenerAdapter`	`junit-platform-launcher/.../ExecutionListenerAdapter.java:28-74`
Subject (notify)	`org.junit.platform.engine.support.hierarchical.NodeTestTask#executeRecursively`	`junit-platform-engine/.../NodeTestTask.java:157-189`
Subject (notify)	`org.junit.platform.engine.support.hierarchical.NodeTestTask#reportCompletion`	`junit-platform-engine/.../NodeTestTask.java:195-226`

Evidence: During execution NodeTestTask pushes executionStarted/Skipped/Finished to the EngineExecutionListener from the ExecutionRequest, without knowing who listens. Also part of Area C.

Key code: `taskContext.listener().executionStarted(testDescriptor);` (NodeTestTask.java:158; see NodeTestTask.java:158, 209, 224)

Diagram: `Part1/uml/png/B8.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B9 Node Execution Tasks **Pattern:** Command (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Command	`org.junit.platform.engine.support.hierarchical.HierarchicalTestExecutorService.TestTask`	`junit-platform-engine/.../HierarchicalTestExecutorService.java:88-116`
ConcreteCommand	`org.junit.platform.engine.support.hierarchical.NodeTestTask`	`junit-platform-engine/.../NodeTestTask.java:46-307`
Invoker	`org.junit.platform.engine.support.hierarchical.SameThreadHierarchicalTestExecutorService`	`junit-platform-engine/.../SameThreadHierarchicalTestExecutorService.java:29-50`

Role	Class / interface / function	File and lines
Invoker	`org.junit.platform.engine.support.hierarchical.ForkJoinPoolHierarchicalTestExecutorService`	`junit-platform-engine/../ForkJoinPoolHierarchicalTestExecutorService.java:51-331`

Evidence: Each node's work is packaged as a TestTask object (execute + execution mode + lock) and handed to an executor service that decides when and on which thread to run it.

Key code: `interface TestTask {` (HierarchicalTestExecutorService.java:88; see HierarchicalTestExecutorService.java:88-114; NodeTestTask.java:46)

Diagram: `Part1/uml/png/B9.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

B10 No-op Node **Pattern:** Null Object (Behavioral, not GoF; Woolf, Pattern Languages of Program Design 3, 1998). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
AbstractObject	`org.junit.platform.engine.support.hierarchical.Node`	`junit-platform-engine/.../Node.java:38-381`
NullObject provider	`org.junit.platform.engine.support.hierarchical.NodeUtils#asNode`	`junit-platform-engine/.../NodeUtils.java:25-27`
Client	`org.junit.platform.engine.support.hierarchical.NodeTestTask`	`junit-platform-engine/.../NodeTestTask.java:46-307`

Evidence: A TestDescriptor that is not a Node gets a do-nothing Node from NodeUtils.asNode, so NodeTestTask never needs null checks.

Key code: `static <C extends EngineExecutionContext> Node<C> asNode(TestDescriptor testDescriptor) {` (NodeUtils.java:25; see NodeUtils.java:25-30)

Diagram: `Part1/uml/png/B10.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

Area C: listeners and reporting

C1 Test Execution Listeners **Pattern:** Observer (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 5.

Role	Class / interface / function	File and lines
Observer	`org.junit.platform.launcher.TestExecutionListener`	`junit-platform-launcher/../TestExecutionListener.java:76-203`
ConcreteObserver	`org.junit.platform.launcher.listeners.SummaryGeneratingListener`	`junit-platform-launcher/../SummaryGeneratingListener.java:36-148`

Role	Class / interface / function	File and lines
ConcreteObserver	`org.junit.platform.launcher.listeners.LoggingListener`	`junit-platform-launcher/.../LoggingListener.java:39-132`
ConcreteObserver	`org.junit.platform.launcher.listeners.UniqueIdTrackingListener`	`junit-platform-launcher/.../UniqueIdTrackingListener.java:94-217`
ConcreteObserver	`org.junit.platform.launcher.jfr.FlightRecordingExecutionListener`	`junit-platform-launcher/.../FlightRecordingExecutionListener.java:43-225`
ConcreteObserver	`org.junit.platform.launcher.core.StreamInterceptingTestExecutionListener`	`junit-platform-launcher/.../StreamInterceptingTestExecutionListener.java:35-99`
Subject (attach)	`org.junit.platform.launcher.core.DefaultLauncher#registerTestExecutionListeners`	`junit-platform-launcher/.../DefaultLauncher.java:80-84`
Subject (notify)	`org.junit.platform.launcher.core.CompositeTestExecutionListener#notifyEach`	`junit-platform-launcher/.../CompositeTestExecutionListener.java:104-116`

Evidence: Client-side observers (IDE, summary, logging, unique-id file, JFR, stdout capture) register three ways and are notified of the same run events; notifyEach isolates each listener's failure. Different from B8 (engine side).

Key code: `private static <T extends TestExecutionListener> void notifyEach(List<T> listeners, IterationOrder iterationOrder,` (CompositeTestExecutionListener.java:104; see CompositeTestExecutionListener.java:104-112 (catches each listener's Throwable))

Diagram: `Part1/uml/png/C1.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C2 Launcher Session Notification **Pattern:** Observer (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Observer	`org.junit.platform.launcher.LauncherSessionListener`	`junit-platform-launcher/.../LauncherSessionListener.java:43-67`
Subject	`org.junit.platform.launcher.core.DefaultLauncherSession`	`junit-platform-launcher/.../DefaultLauncherSession.java:35-159`
Multicaster	`org.junit.platform.launcher.listeners.session.CompositeLauncherSessionListener`	`junit-platform-launcher/.../CompositeLauncherSessionListener.java:24-41`

Evidence: LauncherSessionListeners are told when a launcher session opens and closes by DefaultLauncherSession.

Key code: `listener.launcherSessionOpened(this);` (DefaultLauncherSession.java:71; see DefaultLauncherSession.java:71, 87)

Diagram: `Part1/uml/png/C2.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed

(reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C3 Listener Notification Order **Pattern:** Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Strategy + ConcreteStrategies (ORIGINAL, REVERSED)	`org.junit.platform.launcher.core.IterationOrder`	`junit-platform-launcher/../IterationOrder.java:18-35`
Context	`org.junit.platform.launcher.core.CompositeTestExecutionListener#notifyEach`	`junit-platform-launcher/../CompositeTestExecutionListener.java:104-116`
Context	`org.junit.platform.launcher.core.CompositeEngineExecutionListener#notifyEach`	`junit-platform-launcher/../CompositeEngineExecutionListener.java:78-90`

Evidence: The order in which listeners are notified is an interchangeable algorithm: IterationOrder.ORIGINAL for started events, REVERSED for finished events, used by notifyEach.

Key code: `enum IterationOrder {` (IterationOrder.java:18; see IterationOrder.java:18-34)

Diagram: `Part1/uml/png/C3.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C4 Pluggable Log Sink **Pattern:** Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Strategy	`java.util.function.BiConsumer`	java.util.function.BiConsumer (JDK)
ConcreteStrategy factory	`org.junit.platform.launcher.listeners.LoggingListener#forJavaUtilLogging`	`junit-platform-launcher/.../LoggingListener.java:49-51`
ConcreteStrategy factory	`org.junit.platform.launcher.listeners.LoggingListener#forBiConsumer`	`junit-platform-launcher/.../LoggingListener.java:82-84`
Context	`org.junit.platform.launcher.listeners.LoggingListener`	`junit-platform-launcher/.../LoggingListener.java:39-132`

Evidence: LoggingListener's behaviour is fixed but where log lines go is an injected BiConsumer (java.util.logging via forJavaUtilLogging, or any sink via forBiConsumer).

Key code: `public static LoggingListener forJavaUtilLogging(Level logLevel) {` (LoggingListener.java:62; see LoggingListener.java:62-88)

Diagram: `Part1/uml/png/C4.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C5 Test Reporter Adapter **Pattern:** Adapter (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Target	`org.junit.jupiter.api.TestReporter`	`junit-jupiter-api/.../TestReporter.java:47-231`
Adapter	`org.junit.jupiter.engine.extension.DefaultTestReporter`	`junit-jupiter-engine/.../DefaultTestReporter.java:24-47`
Adaptee	`org.junit.jupiter.engine.descriptor.AbstractExtensionContext#publishReportEntry`	`junit-jupiter-engine/.../AbstractExtensionContext.java:141-143`

Evidence: Tests call the simple TestReporter API; DefaultTestReporter implements it by calling ExtensionContext.publishReportEntry, which turns the map into a ReportEntry event.

Key code: `public void publishEntry(Map<String, String> map) {` (DefaultTestReporter.java:33; see DefaultTestReporter.java:33-34)

Diagram: `Part1/uml/png/C5.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C6 Captured Output Stream **Pattern:** Decorator (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Component	`java.io.PrintStream`	java.io.PrintStream (JDK)
ConcreteDecorator	`org.junit.platform.launcher.core.StreamInterceptor`	`junit-platform-launcher/../StreamInterceptor.java:27-145`
Installer / client	`org.junit.platform.launcher.core.StreamInterceptingTestExecutionListener`	`junit-platform-launcher/../StreamInterceptingTestExecutionListener.java:35-99`

Evidence: StreamInterceptor extends PrintStream, wraps System.out/err, copies every write into a per-thread buffer and still forwards to the original stream (super.write).

Key code: `public void write(int b) {` (StreamInterceptor.java:84; see StreamInterceptor.java:84-90 (copies then super.write))

Diagram: `Part1/uml/png/C6.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

C7 No-op Listeners **Pattern:** Null Object (Behavioral, not GoF; Woolf, Pattern Languages of Program Design 3, 1998). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
NullObject holder	`org.junit.platform.engine.EngineExecutionListener`	`junit-platform-engine/.../EngineExecutionListener.java:34-164`
NullObject holder	`org.junit.platform.launcher.LauncherSessionListener`	`junit-platform-launcher/.../LauncherSessionListener.java:43-67`
Client	`org.junit.platform.launcher.listeners.session.LauncherSessionListeners`	`junit-platform-launcher/.../LauncherSessionListeners.java:28-45`

Evidence: EngineExecutionListener.NOOP and LauncherSessionListener.NOOP stand in for 'no listener' so callers never check for null; LauncherSessionListeners returns NOOP for an empty list.

Key code: `if (listeners.isEmpty()) {` (LauncherSessionListeners.java:36; see LauncherSessionListeners.java:36-37)

Diagram: `Part1/uml/png/C7.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

Area D: Jupiter extensions and resources

D1 Temp Directory Creation Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Strategy	`org.junit.jupiter.api.io.TempDirFactory`	`junit-jupiter-api/.../TempDirFactory.java:49-100`
ConcreteStrategy	`org.junit.jupiter.api.io.TempDirFactory.Standard`	`junit-jupiter-api/.../TempDirFactory.java:83-98`
Selection	`org.junit.jupiter.engine.extension.TempDirectory#determineTempDirFactory`	`junit-jupiter-engine/.../TempDirectory.java:194-200`
Context	`org.junit.jupiter.engine.extension.TempDirectory`	`junit-jupiter-engine/.../TempDirectory.java:72-359`

Evidence: How a @TempDir directory is created is a TempDirFactory chosen from @TempDir(factory) or the configuration; TempDirFactory.Standard is the default.

Key code: `private TempDirFactory determineTempDirFactory(TempDir tempDir) {` (TempDirectory.java:194; see TempDirectory.java:194-199)

Diagram: `Part1/uml/png/D1.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D2 Temp Directory Deletion Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Strategy	`org.junit.jupiter.api.io.TempDirDeletionStrategy`	`junit-jupiter-api/.../TempDirDeletionStrategy.java:71-474`
ConcreteStrategy	`org.junit.jupiter.api.io.TempDirDeletionStrategy.IgnoreFailures`	`junit-jupiter-api/.../TempDirDeletionStrategy.java:97-149`
ConcreteStrategy	`org.junit.jupiter.api.io.TempDirDeletionStrategy.Standard`	`junit-jupiter-api/.../TempDirDeletionStrategy.java:165-360`
Selection	`org.junit.jupiter.engine.extension.TempDirectory#determineDeletionStrategy`	`junit-jupiter-engine/.../TempDirectory.java:187-192`

Evidence: How a temp directory is deleted is a TempDirDeletionStrategy (IgnoreFailures or Standard) chosen from @TempDir(deletionStrategy) or the configuration and called from Cleanup.run.

Key code: `private Supplier<TempDirDeletionStrategy> determineDeletionStrategy(TempDir annotation) {` (TempDirectory.java:187; see TempDirectory.java:187, 349)

Diagram: `Part1/uml/png/D2.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D3 Parameter Resolution **Pattern:** Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Strategy	`org.junit.jupiter.api.extension.ParameterResolver`	`junit-jupiter-api/.../ParameterResolver.java:69-116`
ConcreteStrategy	`org.junit.jupiter.engine.extension.TempDirectory`	`junit-jupiter-engine/.../TempDirectory.java:72-359`
ConcreteStrategy	`org.junit.jupiter.engine.extension.TestInfoParameterResolver`	`junit-jupiter-engine/.../TestInfoParameterResolver.java:30-100`
ConcreteStrategy	`org.junit.jupiter.engine.extension.TestReporterParameterResolver`	`junit-jupiter-engine/.../TestReporterParameterResolver.java:23-40`
Context	`org.junit.jupiter.engine.execution.ParameterResolutionUtils#resolveParameter`	`junit-jupiter-engine/.../ParameterResolutionUtils.java:127-180`

Evidence: For each parameter the context asks every ParameterResolver supportsParameter; exactly one must match and its resolveParameter supplies the value (none or several is an error).

Key code: `List<ParameterResolver> matchingResolvers = extensionRegistry.stream(ParameterResolver.class)` (ParameterResolutionUtils.java:132; see ParameterResolutionUtils.java:132-155)

Diagram: `Part1/uml/png/D3.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D4 Conditional Execution Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Strategy	`org.junit.jupiter.api.extension.ExecutionCondition`	`junit-jupiter-api/.../ExecutionCondition.java:58-73`
ConcreteStrategy	`org.junit.jupiter.engine.extension.DisabledCondition`	`junit-jupiter-engine/.../DisabledCondition.java:31-54`
Context	`org.junit.jupiter.engine.execution.ConditionEvaluator#evaluate`	`junit-jupiter-engine/.../ConditionEvaluator.java:48-59`
Caller	`org.junit.jupiter.engine.descriptor.JupiterTestDescriptor#shouldBeSkipped`	`junit-jupiter-engine/.../JupiterTestDescriptor.java:227-232`

Evidence: Each ExecutionCondition encapsulates an enable/disable policy; ConditionEvaluator applies all of them and the first 'disabled' wins.

Key code: `return extensionRegistry.stream(ExecutionCondition.class)` (ConditionEvaluator.java:52; see ConditionEvaluator.java:52-57)

Diagram: `Part1/uml/png/D4.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D5 Store Close Action Pattern: Strategy (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Strategy	`org.junit.platform.engine.support.store.NamespacedHierarchicalStore.CloseAction`	`junit-platform-engine/.../NamespacedHierarchicalStore.java:734-761`
ConcreteStrategy (lambda)	`org.junit.jupiter.engine.descriptor.AbstractExtensionContext#createCloseAction`	`junit-jupiter-engine/.../AbstractExtensionContext.java:103-123`
Context	`org.junit.platform.engine.support.store.NamespacedHierarchicalStore#close`	`junit-platform-engine/.../NamespacedHierarchicalStore.java:135-165`

Evidence: The platform store does not know how to close Jupiter values, so AbstractExtensionContext injects a CloseAction (close AutoCloseable / CloseableResource values).

Key code: `this.valuesStore = new NamespacedHierarchicalStore<>(getParentStore(parent), createCloseAction());` (AbstractExtensionContext.java:92; see AbstractExtensionContext.java:92-120)

Diagram: `Part1/uml/png/D5.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D6 Extension Store Adapter Pattern: Adapter (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Target	`org.junit.jupiter.api.extension.ExtensionContext.Store`	`junit-jupiter-api/.../ExtensionContext.java:526-897`
Adapter	`org.junit.jupiter.engine.execution.NamespaceAwareStore`	`junit-jupiter-engine/.../NamespaceAwareStore.java:31-127`
Adaptee	`org.junit.platform.engine.support.store.NamespacedHierarchicalStore`	`junit-platform-engine/.../NamespacedHierarchicalStore.java:56-763`
Adapter factory	`org.junit.jupiter.engine.execution.LauncherStoreFacade#getStoreAdapter`	`junit-jupiter-engine/.../LauncherStoreFacade.java:46-50`
Client	`org.junit.jupiter.engine.descriptor.AbstractExtensionContext#getStore`	`junit-jupiter-engine/.../AbstractExtensionContext.java:219-221`

Evidence: Extensions use the namespace-scoped ExtensionContext.Store; NamespaceAwareStore implements it on top of the platform NamespacedHierarchicalStore, fixing the namespace and translating exceptions.

Key code: `Supplier<@Nullable Object> action = () -> this.valuesStore.get(this.namespace, key);` (NamespaceAwareStore.java:44; see NamespaceAwareStore.java:44, 122-123)

Diagram: `Part1/uml/png/D6.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D7 Lifecycle Method Adapters **Pattern:** Adapter (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 0.

Role	Class / interface / function	File and lines
Target	`org.junit.jupiter.engine.execution.BeforeEachMethodAdapter`	`junit-jupiter-engine/.../BeforeEachMethodAdapter.java:29-33`
Target	`org.junit.jupiter.engine.execution.AfterEachMethodAdapter`	`junit-jupiter-engine/.../AfterEachMethodAdapter.java:29-33`
Adapter (lambda)	`org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor#synthesizeBeforeEachMethodAdapter`	`junit-jupiter-engine/.../ClassBasedTestDescriptor.java:533-536`
Adapter (lambda)	`org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor#synthesizeAfterEachMethodAdapter`	`junit-jupiter-engine/.../ClassBasedTestDescriptor.java:538-541`
Client	`org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor#invokeBeforeEachMethods`	`junit-jupiter-engine/.../TestMethodTestDescriptor.java:187-197`

Evidence: A user's @BeforeEach/@AfterEach Method is wrapped as a synthetic extension implementing BeforeEachMethodAdapter/AfterEachMethodAdapter so it runs through the extension ordering. Adapters are lambdas (no named class).

Key code: `registrar.registerSyntheticExtension(extensionSynthesizer.apply(method), method);` (ClassBasedTestDescriptor.java:529; see ClassBasedTestDescriptor.java:529-535)

Diagram: `Part1/uml/png/D7.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D8 Test Outcome Watchers **Pattern:** Observer (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Observer	`org.junit.jupiter.api.extension.TestWatcher`	`junit-jupiter-api/.../TestWatcher.java:70-124`
ConcreteObserver	`org.junit.jupiter.engine.extension.RepetitionExtension`	`junit-jupiter-engine/.../RepetitionExtension.java:41-78`
Subject (notify)	`org.junit.jupiter.engine.descriptor.MethodBasedTestDescriptor#invokeTestWatchers`	`junit-jupiter-engine/.../MethodBasedTestDescriptor.java:166-192`
Subject (trigger)	`org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor#nodeFinished`	`junit-jupiter-engine/.../TestMethodTestDescriptor.java:278-291`

Evidence: TestWatcher extensions are told each test's outcome and cannot change it; their exceptions are only logged.

Key code: `List<TestWatcher> watchers = context.getExtensionRegistry().getExtensions(TestWatcher.class);` (MethodBasedTestDescriptor.java:169; see MethodBasedTestDescriptor.java:169-187)

Diagram: `Part1/uml/png/D8.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D9 Invocation Interceptor Chain **Pattern:** Chain of Responsibility (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 1.

Role	Class / interface / function	File and lines
Handler	`org.junit.jupiter.api.extension.InvocationInterceptor`	`junit-jupiter-api/.../InvocationInterceptor.java:64-253`
ConcreteHandler	`org.junit.jupiter.engine.extension.TimeoutExtension`	`junit-jupiter-engine/.../TimeoutExtension.java:36-217`
Chain builder	`org.junit.jupiter.engine.execution.InvocationInterceptorChain#chainInterceptors`	`junit-jupiter-engine/.../InvocationInterceptorChain.java:52-62`
Link to next handler	`org.junit.jupiter.engine.execution.InvocationInterceptorChain.InterceptedInvocation`	`junit-jupiter-engine/.../InvocationInterceptorChain.java:94-106`
Client	`org.junit.jupiter.engine.execution.InterceptingExecutableInvoker`	`junit-jupiter-engine/.../InterceptingExecutableInvoker.java:40-137`

Evidence: Every test/lifecycle/constructor call passes through a chain of InvocationInterceptors built back to front; each handler can proceed(), skip() or wrap the rest of the chain (interceptor-chain variant).

Key code: `ListIterator<InvocationInterceptor> iterator = interceptors.listIterator(interceptors.size());` (InvocationInterceptorChain.java:56; see InvocationInterceptorChain.java:56-59)

Diagram: `Part1/uml/png/D9.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D10 Timeout Invocation Wrappers **Pattern:** Decorator (Structural, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 3.

Role	Class / interface / function	File and lines
Component	`org.junit.jupiter.api.extension.InvocationInterceptor.Invocation`	`junit-jupiter-api/.../InvocationInterceptor.java:231-251`
ConcreteDecorator	`org.junit.jupiter.engine.extension.SameThreadTimeoutInvocation`	`junit-jupiter-engine/.../SameThreadTimeoutInvocation.java:26-99`
ConcreteDecorator	`org.junit.jupiter.engine.extension.SeparateThreadTimeoutInvocation`	`junit-jupiter-engine/.../SeparateThreadTimeoutInvocation.java:24-52`
ConcreteDecorator	`org.junit.jupiter.engine.execution.InvocationInterceptorChain.ValidatingInvocation`	`junit-jupiter-engine/.../InvocationInterceptorChain.java:108-151`
Client	`org.junit.jupiter.engine.extension.TimeoutExtension#decorate`	`junit-jupiter-engine/.../TimeoutExtension.java:178-190`

Evidence: TimeoutExtension.decorate wraps the Invocation in a timeout-aware Invocation that keeps the original as delegate and adds timing around delegate.proceed(); ValidatingInvocation adds a call-once check.

Key code: `private final Invocation<T> delegate;` (SameThreadTimeoutInvocation.java:28; see SameThreadTimeoutInvocation.java:28, 51)

Diagram: `Part1/uml/png/D10.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

D11 Packaged Reflective Calls **Pattern:** Command (Behavioral, GoF). **Origin:** JUnit framework code (not student-added). **Concrete participant classes:** 2.

Role	Class / interface / function	File and lines
Command	`org.junit.jupiter.api.extension.InvocationInterceptor.Invocation`	`junit-jupiter-api/.../InvocationInterceptor.java:231-251`
ConcreteCommand	`org.junit.jupiter.engine.execution.MethodInvocation`	`junit-jupiter-engine/.../MethodInvocation.java:25-64`

Role	Class / interface / function	File and lines
ConcreteCommand	`org.junit.jupiter.engine.execution.ConstructorInvocation`	`junit-jupiter-engine/.../ConstructorInvocation.java:25-60`
Client	`org.junit.jupiter.engine.execution.InterceptingExecutableInvoker`	`junit-jupiter-engine/.../InterceptingExecutableInvoker.java:40-137`

Evidence: A resolved reflective call (method or constructor + target + arguments) is packaged as an Invocation object so interceptors can run it later, on another thread, or skip it.

Key code: `MethodInvocation<T> invocation = new MethodInvocation<>(method, optionalTarget, arguments);` (InterceptingExecutableInvoker.java:97; see InterceptingExecutableInvoker.java:97; MethodInvocation.java:59-61)

Diagram: `Part1/uml/png/D11.png` (source in `Part1/uml/`, Appendix A). **Verification:** confirmed (reviewer: Gautham Mohanraj); claims and line numbers checked in `Part1/evidence/verification.csv`, decisions in `Part1/notes.md`.

Table 37: Rejected and uncertain candidates (reasons from `Part1/notes.md`)

Area	Verdict	Candidate and reason
A	rejected	AbstractTestDescriptor – not Template Method: skeletal implementation, final getters (lines 120/140/150), no method calls an abstract step.
A	rejected	TagFilter – not a Strategy itself: final static factory; the lambdas it returns are the strategies.
A	rejected	TestDescriptor.Visitor – not GoF Visitor: one visit() method, no double dispatch.
A	rejected	CompositeTestDescriptorVisitor – not Composite (see decision above).
A	rejected	DelegatingLauncherDiscoveryRequest – not Adapter: same interface wrapped and forwarded (Decorator-style).
A	rejected	TestIdentifier – not Adapter/Proxy: immutable snapshot, keeps no TestDescriptor reference (TestIdentifier.java:49-67).
A	uncertain	Filter.adaptFilter – uncertain Adapter: deprecated for removal, not used.
B	uncertain	NodeTestTask + Node (Node lifecycle hooks) – UNCERTAIN Template Method: fixed step order, but the skeleton calls a separate Node object (delegation), not subclass hooks.
B	rejected	TestEngine – not Strategy: plug-in SPI; the orchestrator runs every engine (EngineExecutionOrchestrator.java:188), none is chosen.
B	rejected	ParallelHierarchicalTestExecutorServiceFactory – not Factory Method: static factory with private constructor and switch (lines 86-93).
B	rejected	ClosedLauncher – not Null Object: every method throws (fail-fast sentinel).
B	rejected	EngineExecutionOrchestrator – not Mediator: controller running a fixed sequence.
B	rejected	LauncherStoreFacade – not Facade: holds two stores and creates NamespaceAwareStore adapters.
B	rejected	HierarchicalTestExecutor – not Strategy/Facade: one-shot coordinator that builds and submits the root task.
C	rejected	Template Method – rejected for listeners/TestReporter/TestExecutionSummary: only optional callbacks and convenience overloads, no skeleton.
C	uncertain	FlightRecordingExecutionListener, LoggingListener as Adapter – UNCERTAIN: they are observers that use JFR / logging.
C	uncertain	ListenerRegistry composite factory as Strategy – UNCERTAIN (see above).

Table 37: Rejected and uncertain candidates (reasons from `Part1/notes.md`)

Area	Verdict	Candidate and reason
C	rejected	ListenerRegistry / LauncherListenerRegistry – not Registry/Service Locator: per-run list builders, no lookup by key.
C	rejected	MutableTestExecutionSummary – not Builder: read-only interface over a mutable object.
C	rejected	ReportEntry – not Builder: value object with static factory methods.
C	rejected	LoggingListener.forJavaUtilLogging/forBiConsumer – not Factory Method: static factories.
C	rejected	StreamInterceptingTestExecutionListener – not Interceptor: an (eager) observer.
C	rejected	UniqueIdTrackingListener – not State: a boolean flag.
D	uncertain	Lifecycle callbacks (BeforeEachCallback, AfterEachCallback, ...) as Observer – UNCERTAIN: they can fail the test or inject fields, so they are hooks rather than passive observers (CallbackSupport.java:33-36).
D	uncertain	NamespacedHierarchicalStore parent lookup as Chain of Responsibility – UNCERTAIN: CoR-shaped, but really a scoped lookup (lines 465-471).
D	uncertain	AbstractExtensionContext.getExecutionMode -> getPlatformExecutionMode – UNCERTAIN Template Method: only one step, not counted.
D	rejected	CallbackSupport – not Template Method: static helper taking a lambda.
D	rejected	ExtensionContext / NamespacedHierarchicalStore / MutableExtensionRegistry – not Composite: parent links only, no children.
D	rejected	TempDirFactory – not Factory Method (it is a creation strategy, D1); TimeoutInvocationFactory – simple factory.
D	rejected	CleanupMode – not Strategy/State: enum flag checked with an if.
D	rejected	ConditionEvaluator, ParameterResolutionUtils – not Chain of Responsibility: they loop over strategies.
D	rejected	AutoCloseExtension – not Adapter: a callback extension; CloseablePath – not Proxy: resource holder.
C	uncertain	CompositeTestExecutionListener, CompositeEngineExecutionListener, CompositeLauncherSessionListener as Composite: flat multicasters that only broadcast (Composite rule; recorded in the Area C decisions)

3 Part 2 Evolution and refactoring

I first asked the LLM what JUnit already offers for each requested change and what the gap is (`Part2/llm/prompt_1_design.txt`); I verified its answer (22 claims, `Part2/evidence/verification.csv`: 111 of 112 line references exact, 3 design claims corrected). The changes were then implemented on branch `b16-evolution` of a JUnit fork (its five commits are exported in `Part2/changes/b16-evolution.bundle`, which applies on top of the public tag `r6.1.3`; the final source is also saved in `Part2/code/`), in the order CH03, CH01, CH02, each with its refactoring as a separate commit, and JUnit's own test suites were rerun after every commit (`Part2/changes/launcher_tests_by_snapshot.txt`). Each change folder contains the full record (`Part2/changes/ChangeN/README.md`), the diffs, test logs and a demo log that runs the same tests on the official 6.1.3 jar and on the evolved jar.

CH03: configurable filtering policy for test selection

Change and acceptance criteria: Existing feature: `PostDiscoveryFilters` such as `TagFilter` and `MethodFilter`, but they can only be built in code; no filter can be set from `junit-platform.properties`. Extension: one policy, `junit.platform.filter.policy.mode=off|exclude|only`, applied to `.tags` (tag expressions)

and/or `.methods` (regexes), matched with OR. Criteria: off/unset changes nothing; `exclude` removes matching tests and prunes empty classes; `only` keeps them; the policy only narrows other filters; invalid values fail discovery with a message naming the key.

Before and after: `f59f60d` → `68a60e4` (refactoring) → `6f98fbf`. New `junit-platform-launcher/.../launcher/core/FilterPolicy.java`; edited `LauncherConstants.java` (3 keys) and `EngineDiscoveryOrchestrator.java`. This is an **implementation change**: a `ServiceLoader` filter cannot see the request's configuration, and the only workaround (static state captured by a discovery listener) is unsafe with concurrent launchers.

Implementation and tests: `FilterPolicy.fromConfiguration` parses and validates everything once per discovery and is added *last* in the new `collectPostDiscoveryFilters`; `apply` only evaluates predicates, so it cannot throw (the orchestrator does not catch filter exceptions). `FilterPolicyTests`: 9 tests (one per criterion, 4 invalid configurations). Launcher tests 382 → 391, all platform tests 2452 → 2461, 0 failures; with the wiring line removed, 8 of the 9 new tests fail (mutation check). Execution order (demo log, `Part2/changes/Change3/demo_ch03.log`): discovery → ServiceLoader filters, request filters, policy → AND-composition and removal of excluded leaves (logged as "2 tests were excluded by filter policy") → pruning → `TestPlan` → execution. Filtered tests are absent, not skipped. Failure handling: invalid mode, missing tags/methods, invalid tag expression or regex → `JUnitException` naming the key, no test runs (console exit 255).

Pattern effects: A3 Post-Discovery Filter Policies (Strategy): **extended** (new ConcreteStrategy `FilterPolicy`, new context method). A2 Composed Discovery Filter (Composite): **extended** (new Leaf, `CompositeFilter` unchanged). All others preserved; A1 and A6 only moved lines. Not new patterns: `FilterPolicy` ORs `Predicate`s, not filters of its own interface (fails the Composite rule), and its mode is a boolean flag, not a Strategy (the LLM's "new enum Strategy" claim was rejected, D1-18).

Refactoring: Extract Method `collectPostDiscoveryFilters` out of `discoverSafely`, as a *preparatory* refactoring (make the change easy, then make the easy change). The LLM called it a Long Method; metrics do not support that (24 NLOC, CCN 3), so the claim was corrected (D1-20). Behaviour preserved: launcher tests 382/382 before and after the refactoring commit.

Before/after metrics: lizard 1.24.1, same files: `discoverSafely` 24 → 22 NLOC (CCN 3 → 3); `EngineDiscoveryOrchestrator.java` 202 → 206 NLOC, 18 → 19 functions; new `FilterPolicy.java` 74 NLOC, max CCN 5 (`Part2/evidence/metrics_before.txt`, `metrics_after_ch03.txt`).

CH01: test execution listener with a custom summary

Change and acceptance criteria: Existing feature: `SummaryGeneratingListener`, which gives one set of totals for the whole run, is not auto-registered and has no settings. Extension: `ClassSummaryListener` writes one row per test class (total, passed, failed, aborted, skipped, time) and the N slowest tests to a file in the reporting output directory; keys `junit.platform.listeners.summary.enabled` (default false), `.slowest` (5), `.output.file`. Criteria: off by default; correct per-class rows whose totals equal `SummaryGeneratingListener`'s; slowest tests in descending order; bad settings and I/O errors are logged and never affect the run.

Before and after: `6f98fbf` → `67d159b` (refactoring) → `e55fefe`. New `junit-platform-launcher/.../listeners/ClassSummaryListener.java`; one line each in `META-INF/services/org.junit.platform.launcher.TestExecutionListener` and `module-info.java`. This uses an **existing extension point** (`TestExecutionListener` + `ServiceLoader`); no JUnit behaviour changes.

Implementation and tests: State is kept in concurrent maps and atomic counters because callbacks can come from several threads; a test's class is found by walking up to the nearest method or class source. `ClassSummaryListenerTests`: 5 tests, run through `LauncherFactory` so the ServiceLoader registration is tested too. Launcher tests 391 → 397, all platform tests 2461 → 2467, 0 failures; mutation checks (registration removed: 4/5 fail; skipped tests not counted: row test fails). Execution order (`Part2/changes/Change1/demo_ch01.log`, with a demo trace listener):

`testPlanExecutionStarted` reads settings → per test `executionStarted`/`executionFinished` (or `executionSkipped`) → `testPlanExecutionFinished` writes the file. Failure handling: `slowest=abc` → warning, default used; unwritable file → error logged, same test results and exit code; an exception inside the listener is isolated by `CompositeTestExecutionListener.notifyEach`. The standard deactivation key only works as a JVM system property, not as a request parameter (found during the demo; D1-21).

Pattern effects: C1 Test Execution Listeners (Observer): **extended** (new ConcreteObserver; the ConcreteObserver `SummaryGeneratingListener` was refactored internally, role unchanged). C3, B7, B8 and all others preserved. "Disabled" is an `enabled` flag, not a Null Object (rejected).

Refactoring: Duplicated Code in `SummaryGeneratingListener`: the "if container ...; if test ..." counting pair appeared five times and the skipped-descendants stream was built twice. Extract Method `count(identifier, containers, tests)`, two separate `if`s kept because a `CONTAINER_AND_TEST` node counts as both. Because existing tests did not cover skipped descendants or `CONTAINER_AND_TEST`, a characterization test was added first and shown to pass on the *original* code (16/16); it also catches the tempting `if/else` mistake.

Before/after metrics: `SummaryGeneratingListener.java`: 104 → 78 NLOC, average CCN 2.4 → 1.5, `executionFinished` 33 NLOC/CCN 10 → 14/4. New `ClassSummaryListener.java`: 190 NLOC, max CCN 7 (`Part2/evidence/metrics_after_ch01a.txt`, `metrics_after_ch01.txt`).

CH02: Jupiter extension for a managed resource

Change and acceptance criteria: Existing feature: `@TempDir` creates, injects and cleans up a resource, but only for `Path`/`File`; `@AutoClose` only closes a field you created yourself. Extension: `@ManagedResource(Factory.class)` on any method parameter, with `ResourceFactory<T>` (`create`, `destroy`). The lifetime follows the method that asks first: `@BeforeAll` → one per class, test or `@BeforeEach` → one per test. Criteria: per-test and per-class lifetimes with exact create/destroy order; destroyed even when the test fails; failing `create`/`destroy` and invalid use are reported as test failures; works when auto-closing of stored values is disabled.

Before and after: `e55fefe` → `99987b0`. Three new files in `junit-jupiter-api/.../api/extension/support/`; **no existing JUnit file changed**. This uses **existing extension points only**: `ParameterResolver`, `@ExtendWith` as a meta-annotation, and `ExtensionContext.Store` for cleanup.

Implementation and tests: The resource is stored, keyed by its factory class, in the store of the context where it is first requested; the store's parent lookup lets a test reuse a class-wide resource. A nested `Holder` implements both `AutoCloseable` and the deprecated `Store.CloseableResource`, so cleanup also runs when auto-close is disabled. `ManagedResourceTests`: 7 tests; all Jupiter tests 3381 → 3388, 0 failures; mutation checks (fallback removed; wrong store lifetime) are caught. Execution order (`Part2/changes/Change2/demo_ch02.log`, with a real `HttpServer`): class scope: create (while resolving the `@BeforeAll` parameter) → `@BeforeAll` → tests reuse it → `@AfterAll` → destroy; test scope: create → `@BeforeEach` → test → `@AfterEach` → destroy → test reported finished. Failure handling: failed test → still destroyed; `create` throws → `ParameterResolutionException`, nothing to destroy; `destroy` throws → the owning test fails with "Failed to close extension context". The official 6.1.3 jar cannot even compile the demo (missing `ManagedResource`, `ResourceFactory`).

Pattern effects: D3 Parameter Resolution (Strategy): **extended** (new ConcreteStrategy `ManagedResourceExtension`). **Newly introduced**: N1 Managed Resource Factory (Strategy; the ConcreteStrategy role is absent from production code because users supply the factories) and N2 Managed Resource Holder (Adapter: `Holder` adapts `ResourceFactory.destroy(T)` to the `close()` the store expects). The LLM had said `Holder` is "not a pattern"; that claim was corrected (D1-22). D1, D5, D6 and all others preserved.

Refactoring: None, deliberately: CH02 changes no existing code. The nearest smell (two parallel close paths for `AutoCloseable` and the deprecated `CloseableResource`) cannot be removed without changing behaviour for users who disabled auto-close, so it would not be a refactoring.

Before/after metrics: New code only: 88 NLOC, max CCN 3 (`resolveParameter` 12 NLOC). lizard does not analyse methods inside a Java `record`; by its counting rule `Holder.create` is CCN 3 and `close` CCN 1 (`Part2/evidence/metrics_after_ch02.txt`).

3.1 Updated pattern counts

Generated by `data/build_part2_csv.py` from the fork at each commit; one row per instance per version in `Part2/pattern_evolution.csv`, counts for every version in `data/pattern_counts.csv`. Rejected observations grow because candidates considered during the changes were recorded (CH03: `FilterPolicy` as Composite, the mode flag as Strategy; CH01: the `enabled` flag as Null Object).

Snapshot	Types	Instances	Concrete classes	Unique symbols	Rejected / uncertain	Changed instances
Original code (`f59f60d`)	10	35	76	152	25 / 8	–
Code after Change3 (`6f98fbf`)	10	35	78	154	27 / 8	A2 extended, A3 extended
Code after Change1 (`e55fefe`)	10	35	79	155	28 / 8	C1 extended
Code after Change2 (`99987b0`)	10	37	81	163	28 / 8	D3 extended, N1 newly introduced, N2 newly introduced

Interpretation: All three changes reuse JUnit's existing patterns instead of adding new mechanisms: CH03 and CH01 add one more concrete Strategy or Observer, which is exactly what those patterns are for (open for extension, closed for modification); only CH03 needed a small change to JUnit's internals, and the preparatory refactoring kept that change to one line. CH02 shows the cost side: generalising `@TempDir` needed two small new collaborations (N1, N2), and it has to support JUnit's two parallel cleanup paths. The refactorings reduced complexity where the smell was real (CH01: CCN 10 $\rightarrow$ 4) and changed almost nothing where it was only preparatory (CH03), which is why the report does not claim a design improvement from lines of code alone. Limitations: CH03 values are split on commas, so a method regex cannot contain a comma; CH01 groups only tests, not containers; CH02 does not support constructor parameters.

4 Part 3 Cross-language comparison

Selected instances: One representative, verified instance for each of the 10 confirmed pattern types, so every detected type has a small problem in all four languages (Table 39). Each instance was chosen because its behaviour can be reduced to a deterministic text input and output without JUnit itself, and because it is the clearest example of its type in the four areas.

Table 39: Part 3 problems and the Part 1 instance each one models

Pattern	Source instance	Folder and problem
Observer	C1 Test Execution Listeners	`observer-test-listeners/`: listeners notified of test events; a throwing listener is isolated
Composite	A1 Test Descriptor Tree	`composite-test-tree/`: tree of engine, classes, tests; remove tagged tests and prune

Table 39: Part 3 problems and the Part 1 instance each one models

Pattern	Source instance	Folder and problem
Strategy	A3 Post-Discovery Filter Policies	`strategy-test-filters/`: three interchangeable test filters applied by one selector
Adapter	B7 Engine-to-Launcher Listener Adapter	`adapter-listener-bridge/`: engine events with unique ids converted for a client listener
Decorator	D10 Timeout Invocation Wrappers	`decorator-timeout-invocation/`: timeout and logging wrappers around a test invocation
Template Method	B1 Hierarchical Engine Execution Skeleton	`template-method-engine/`: fixed `execute` with a primitive and two hooks
Command	B9 Node Execution Tasks	`command-task-queue/`: task objects queued and run later by an executor
Null Object	C7 No-op Listeners	`null-object-listener/`: a no-op listener instead of null checks
Chain of Responsibility	D9 Invocation Interceptor Chain	`chain-of-responsibility-interceptors/`: interceptors that pass on or stop an invocation
Factory Method	B2 Engine Collaborator Factory Methods	`factory-method-executors/`: engines that create their own executor

Common specifications: Every problem has a `test-data/` folder shared by all four languages: an input file, the exact `expected-output.txt` (LF, trailing newline), an invalid input and the exact message it must produce (`expected-error.txt`). Common rules: blank and `#` lines are ignored, fields are split on whitespace, durations are given in the input (not measured) so output is deterministic, and a bad line stops the program with `error: line N: ...` on stderr and exit code 1. Each test runner checks three things: the exact output, the exact error for the bad input, and one property specific to the pattern (for example: a new strategy plugs in unchanged; decorators nest in any order; a handler can stop the chain; a subclass changes only the created product). The rules for each problem are in its `test-data/README.md`.

Implementations: `Part3/<problem>/{java,python,javascript,cpp}/`, each with production code separate from its test runner and a README giving the commands (run from that folder) and how the pattern looks in that language. Versions: OpenJDK 25.0.4.1, Python 3.14.2, Node.js 26.11.1, Apple clang 21.0.0 `-std=c++20`; standard library only. All commands: `sh run-all.sh` in `Part3/`. Observer and Composite were generated on claude.ai; the other eight by Claude Code (recorded in `data/llm.csv`).

Actual results: On my Mac: 110/110 compile, run and test commands exit 0 and all 40 programs' output is byte-identical to the expected file (`Part3/RUN-LOG-mac.txt`); every one of the 40 test runners passes 3/3, with one log per program in `Part3/evidence/test_results/`; 0 compiler warnings (`clang++ -Wall -Wextra -pedantic`; Java also clean under `javac -Xlint:all -Werror`); all ten C++ test runners are clean under AddressSanitizer and UndefinedBehaviorSanitizer and macOS `leaks` reports 0 leaks (`Part3/cpp-sanitizer-mac.txt`). Nothing was marked not run.

Static and dynamic comparison: Measured by `Part3/measure.py` into `data/cross_language.csv`. Source LOC = lizard NLOC of production files only; max CC = highest lizard CCN of any production function; time = median wall-clock time of the whole process over 5 runs after 1 warm-up, including runtime start-up (raw values in `Part3/timing-raw.csv`). Summary per language over the 10 problems, then every program:

Language	Mean source LOC	LOC range	Max CC range	Median ms range
Java	138.0	98–228	6–22	32.0–36.8
Python	86.5	57–141	6–22	21.0–21.8
JavaScript	102.0	70–151	5–17	91.4–93.2
C++	142.1	110–236	6–24	4.8–5.9

Per program (one test log each in `Part3/evidence/test_results/`, e.g. `Observer_Java.txt`, `TemplateMethod_Cpp.txt`):

Pattern	Language	Source LOC	Max function CC	Tests passed	Median ms
Observer	Java	211	7	3/3 passed	36.0
Observer	Python	125	8	3/3 passed	21.8
Observer	JavaScript	151	8	3/3 passed	93.2
Observer	C++	236	10	3/3 passed	5.1
Composite	Java	228	22	3/3 passed	34.0
Composite	Python	141	22	3/3 passed	21.7
Composite	JavaScript	135	17	3/3 passed	92.2
Composite	C++	207	24	3/3 passed	5.1
Strategy	Java	104	6	3/3 passed	32.8
Strategy	Python	77	7	3/3 passed	21.2
Strategy	JavaScript	108	5	3/3 passed	92.8
Strategy	C++	122	7	3/3 passed	5.1
Adapter	Java	128	10	3/3 passed	33.6
Adapter	Python	82	10	3/3 passed	21.1
Adapter	JavaScript	97	9	3/3 passed	92.4
Adapter	C++	124	12	3/3 passed	5.9
Decorator	Java	128	7	3/3 passed	34.9
Decorator	Python	88	7	3/3 passed	21.2
Decorator	JavaScript	98	6	3/3 passed	92.7
Decorator	C++	132	7	3/3 passed	5.0
Template Method	Java	113	6	3/3 passed	36.8
Template Method	Python	70	6	3/3 passed	21.5
Template Method	JavaScript	96	5	3/3 passed	92.3
Template Method	C++	114	6	3/3 passed	5.1
Command	Java	104	11	3/3 passed	34.2
Command	Python	69	10	3/3 passed	21.2
Command	JavaScript	83	10	3/3 passed	92.6
Command	C++	120	12	3/3 passed	5.6
Null Object	Java	98	6	3/3 passed	32.1
Null Object	Python	57	6	3/3 passed	21.0
Null Object	JavaScript	70	5	3/3 passed	91.7
Null Object	C++	110	6	3/3 passed	5.4
Chain of Responsibility	Java	142	10	3/3 passed	32.0
Chain of Responsibility	Python	82	9	3/3 passed	21.5
Chain of Responsibility	JavaScript	87	9	3/3 passed	91.4
Chain of Responsibility	C++	128	10	3/3 passed	5.6
Factory Method	Java	124	6	3/3 passed	34.4
Factory Method	Python	74	6	3/3 passed	21.5
Factory Method	JavaScript	95	5	3/3 passed	92.8
Factory Method	C++	128	6	3/3 passed	4.8

Comparison: The participants are the same in every language, but the languages express them differently. *Contracts*: Java and C++ state them in the type system (Java `interfaces`, records and

a `final` template method; C++ pure virtual functions, `final` classes and a non-virtual `execute`), so a missing or wrong override fails at compile time. Python uses `abc.ABC` where every method is required (Strategy, Decorator, Template Method, Command, Chain of Responsibility, Composite, Factory Method) and plain base classes where callbacks are optional (Observer, Adapter, Null Object); missing methods are found at run time, when an object is created or called. JavaScript has neither interfaces nor abstract classes: abstract base classes (Strategy, Template Method, Composite, Factory Method) block `new` with `new.target` and throw from unimplemented methods, and the tests pass plain objects as strategies and listeners (duck typing). *Template Method*: only Java (`final`) and C++ (non-virtual) can prevent overriding the skeleton; in Python and JavaScript it is a convention. *Null Object*: in C++ the runner holds a reference, which cannot be null, so the language itself removes the null check; elsewhere a shared no-op object replaces `null`/`None`. *Chain of Responsibility*: all four pass each handler a `proceed` callback (Java `Supplier`, Python lambda, JavaScript arrow function, C++ `std::function`) instead of a fixed `next` field, so handlers can be reordered; JUnit continues its chain through `Invocation.proceed()`. *Decorator*: every wrapper holds the inner invocation and calls its `proceed()`, so wrappers nest in any order. *Ownership* is explicit only in C++: decorators, commands and factory products are held in `std::unique_ptr`, so building or destroying the outer object moves or frees the whole structure, while the other three rely on garbage collection. All Composite versions use the "safe" form (`addChild` only on containers), whereas JUnit's `TestDescriptor` is "transparent". Python is the shortest version for 9 of the 10 problems (JavaScript for Composite), and C++ or Java the longest, mostly because of type declarations, headers and explicit ownership. The highest complexity is in input parsing and event dispatch (Composite parsers CCN 17–24; Observer, Adapter, Command and Chain parsers 7–12), not in the pattern classes. lizard failed to parse one JavaScript file (a template literal nested inside another one); that line was rewritten without nesting and re-measured (`Part3/llm/verification.csv`, P3-16).

Technology discussion: The times are dominated by runtime start-up, not by the pattern code: native C++ binaries run in about 5–6 ms, Python in about 21–22 ms, the JVM in about 32–37 ms and Node.js in about 91–93 ms, almost independently of the pattern (ranges over the 10 problems). In a test framework such as JUnit the JVM start-up is paid once per run, and pattern calls (listener notifications, strategy calls, interceptor hops) are in-process method calls, so their cost is negligible next to process start-up. The static/dynamic contrast matches the usual trade-off described by Gamma et al. (an explicit interface makes the participant roles checkable; duck typing makes them implicit but shorter) and the language documentation (Python `abc`, C++ smart pointers).

5 LLM records and human verification

Run and model: Claude Opus 5.5 on claude.ai (Claude Code on the web), 2026-10-10, for the analysis prompts of Parts 1–3 and the Observer and Composite programs of Part 3; Claude Code CLI (Claude Opus 5.5) on my Mac for implementing the three Part 2 changes, the other eight Part 3 problems, the measurement and verification scripts and the draft of this report, following plans I approved. Every interaction is a row in `data/llm.csv` (prompts combined per part in `Part1/Part1_prompt.txt`, `Part2/Part2_prompt.txt`, `Part3/llm/part3_prompt.txt`).

Step	Tool	Exact prompt	Evaluation
Part 1 – pattern discovery (prompts in `Part1/llm/`)			
Prompt 1: map files and flow per area	claude.ai (Claude Code on the web)	`prompt_1_files.txt`	correct
Prompt 2: simple explanation of the Area A flow	claude.ai (Claude Code on the web)	`prompt_2_areaA_flow.txt`	correct
Prompt 3: patterns in Area A	claude.ai (Claude Code on the web)	`prompt_3_areaA_patterns.txt`	partially correct

Step	Tool	Exact prompt	Evaluation
Prompt 4: simple explanation of the Area B flow	claude.ai (Claude Code on the web)	`prompt_4_areaB_flow.txt`	partially correct
Prompt 4b: follow-up challenging step 7 of prompt 4	claude.ai (Claude Code on the web)	`prompt_4b_correction.txt`	correct
Prompt 5: patterns in Area B	claude.ai (Claude Code on the web)	`prompt_5_areaB_patterns.txt`	correct
Prompt 6: simple explanation of the Area C flow	claude.ai (Claude Code on the web)	`prompt_6_areaC_flow.txt`	correct
Prompt 7: patterns in Area C	claude.ai (Claude Code on the web)	`prompt_7_areaC_patterns.txt`	correct
Prompt 8: simple explanation of the Area D flow	claude.ai (Claude Code on the web)	`prompt_8_areaD_flow.txt`	correct
Prompt 9: patterns in Area D	claude.ai (Claude Code on the web)	`prompt_9_areaD_patterns.txt`	correct
UML (prompt 10: PlantUML class diagram per confirmed instance)	claude.ai (Claude Code on the web)	`prompt_10_uml.txt`	partially correct
Architecture (prompt 11: PlantUML component diagram of one test run)	claude.ai (Claude Code on the web)	`prompt_11_architecture.txt`	partially correct
Part 2 – evolution (prompts in `Part2/llm/`)			
Change design (prompt 1: existing features, gap, design, extension point, order/failures, pattern impact, refactoring for CH01-CH03)	claude.ai (Claude Code on the web)	`prompt_1_design.txt`	partially correct
CH03 implementation (refactoring + FilterPolicy + tests + demo)	Claude Code CLI	`prompt_2_ch03_impl.txt`	correct after review
CH01 implementation (refactoring + ClassSummaryListener + tests + demo)	Claude Code CLI	`prompt_3_ch01_impl.txt`	correct after review
CH02 implementation (@ManagedResource extension + tests + demo)	Claude Code CLI	`prompt_4_ch02_impl.txt`	correct after review
Part 3 – cross-language analysis (prompts in `Part3/llm/`)			
Prompt 1 (Observer and Composite programs in Java, Python, JavaScript, C++)	claude.ai (Claude Code on the web)	`prompt_1_programs.txt`	correct, with 1 partial and 1 not confirmable
Remaining 8 pattern types (Strategy, Adapter, Decorator, Template Method, Command, Null Object, Chain of Responsibility, Factory Method) in Java, Python, JavaScript and C++	Claude Code CLI	`prompt_2_eight_patterns.txt`	correct after review
Reporting and tooling for all parts (report draft, slides, dataset workbook, README, dataset generators, measurement and verification scripts)	Claude Code CLI	`data/prompt_reporting_with_claude_code.txt`	correct after review

Inputs and outputs: Input for all analysis prompts: JUnit at commit `f59f60d` (the four modules) and my own earlier findings; raw outputs are saved unchanged next to each prompt (`Part1/llm/output_*.md`, `Part2/llm/output_1_design.md`, `Part3/llm/output_1_programs.zip`).

Claims and evaluation: Every claim was checked against the source, tests or measurements (scripts and code reading, recorded per row with file and line evidence); the `checked_by` column names the person who re-checked a row by hand.

Verification file	Claims	Correct	Partially	Incorrect	Cannot confirm	Re-checked by hand
Part 1	102	93	6	2	1	8
Part 2	22	15	5	2	0	5
Part 3	16	13	2	0	1	5

Examples of corrections: the LLM claimed no test instance is created for a `@Disabled` method (wrong: `prepare()` runs before `checkWhetherSkipped`); its architecture diagram had the client call the launcher from the request builder and the orchestrator build the `TestPlan` (both corrected); its UML for A2 included `EngineFilter` (removed); in Part 2 it proposed a non-existent enum Strategy, a Long Method that metrics do not support, and called the CH02 `Holder` "not a pattern".

Generated code: Part 2: all production code, tests and demos were generated by Claude Code and accepted only after JUnit's own suites passed, new tests were shown to fail when the feature is removed (mutation checks), and every claim in the change READMEs was checked against logs. Part 3: the Observer and Composite programs were generated on claude.ai and used unchanged; the other 32 programs (eight pattern types) were written by Claude Code. All 40 were accepted only after re-running everything on my machine, sanitizers, leak checks, zero-warning builds and a check of every JUnit fact in their READMEs against the source (`Part3/llm/verification.csv`).

6 Findings, contributions and references

Findings: JUnit 6.1.3 relies on a small set of patterns used many times: Strategy (10 instances) for every variable policy (filters, executors, temp directories, parameter resolution, conditions), Observer (5) for discovery and execution events, Adapter (5) between the engine and launcher APIs, and Decorator (4) around listeners, the launcher and invocations; Composite holds the test tree everything else works on. The extension points built on these patterns made two of the three changes possible without touching JUnit's code, and the third needed one line in the launcher. Limitations: only four modules and only production code were analysed; pattern recognition depends on the stated rules (for example, the `Composite*Listener` classes are uncertain under our Composite rule); timings in Part 3 are single-machine measurements dominated by start-up.

Contributions: AM.SC.U4CSE23319, Gautham Mohanraj, CSE D: whole case study (single-member team): prompts, verification and decisions in all parts, choice and design of the three changes, review of the generated code, measurements and this report; evidence as linked above.

References:

- JUnit 6.1.3 source, `https://github.com/junit-team/junit-framework/tree/r6.1.3`, and the JUnit User Guide, `https://docs.junit.org/6.1.3/`.
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- B. Woolf, "Null Object", in *Pattern Languages of Program Design 3*, Addison-Wesley, 1998.
- M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2018 (Extract Method, Duplicated Code, preparatory refactoring).
- Python `abc` module, `https://docs.python.org/3/library/abc.html`; Node.js test runner, `https://nodejs.org/api/test.html`; C++ smart pointers, `https://en.cppreference.com/w/cpp/memory`.

- Tools: lizard (`https://github.com/terryyin/lizard`), PlantUML (`https://plantuml.com`), Gradle (`https://gradle.org`).

7 Team folder and datasets

Screenshots of the GitHub team folder can only be taken after upload; the structure submitted is:

```
B16/
  README.md, B16_Report.tex, B16_Report.pdf, B16_dataset.xlsx
  data/   patterns.csv, pattern_counts.csv, cross_language.csv, llm.csv
          (the only copy of the four datasets), code/ (baseline source),
          test_result/ (test logs index), generator scripts
  Part1/  Part1_prompt.txt, llm/, evidence/, uml/, architecture/, notes.md
  Part2/  code/ (evolved source + demo/), changes/ (Change1..3/,
          b16-evolution.bundle), pattern_evolution.csv, uml/,
          Part2_prompt.txt, llm/, evidence/
  Part3/  README.md, one folder per pattern type (10):
          observer-test-listeners/, composite-test-tree/,
          strategy-test-filters/, adapter-listener-bridge/,
          decorator-timeout-invocation/, template-method-engine/,
          command-task-queue/, null-object-listener/,
          chain-of-responsibility-interceptors/, factory-method-executors/
          (each: java, python, javascript, cpp, test-data),
          llm/ (part3_prompt.txt, raw output, verification),
          evidence/test_results/, run-all.sh, run logs, measure.py,
          timing-raw.csv, lizard-raw.txt, B16_slide.tex, B16_slide.pdf
```

A UML diagrams

Baseline diagrams for all 35 instances (`Part1/uml/`, PlantUML sources and PNGs; 98 of 98 implements/extends arrows checked against the code), followed by the diagrams that changed in Part 2 (`Part2/uml/`; new participants in green).

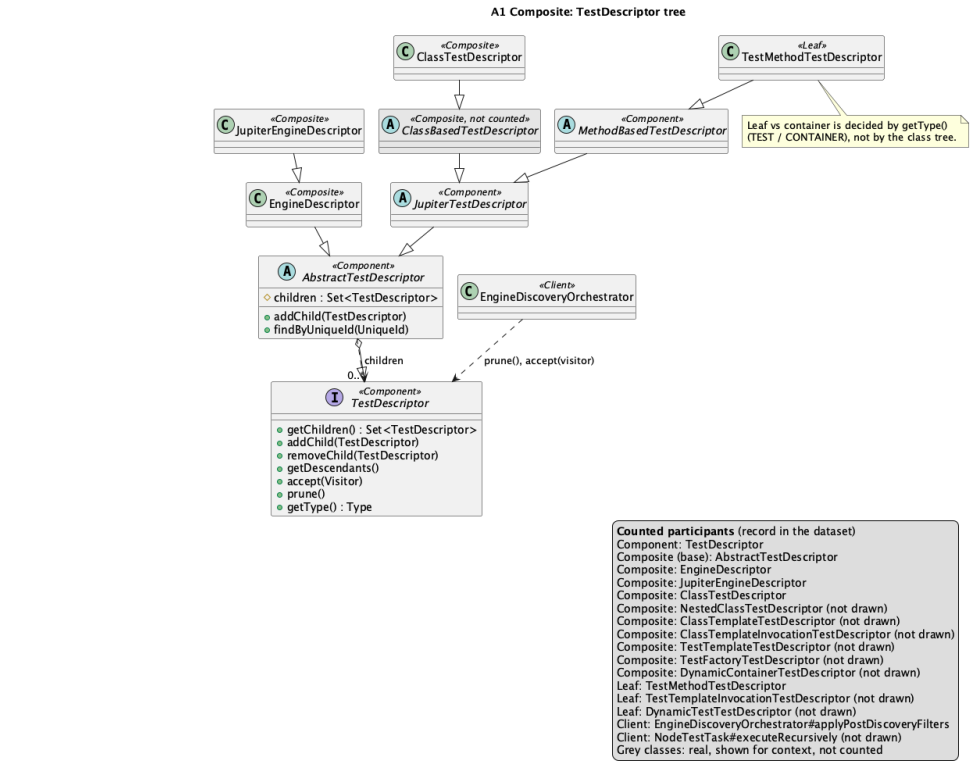


Figure 2: A1 Test Descriptor Tree (Composite), baseline

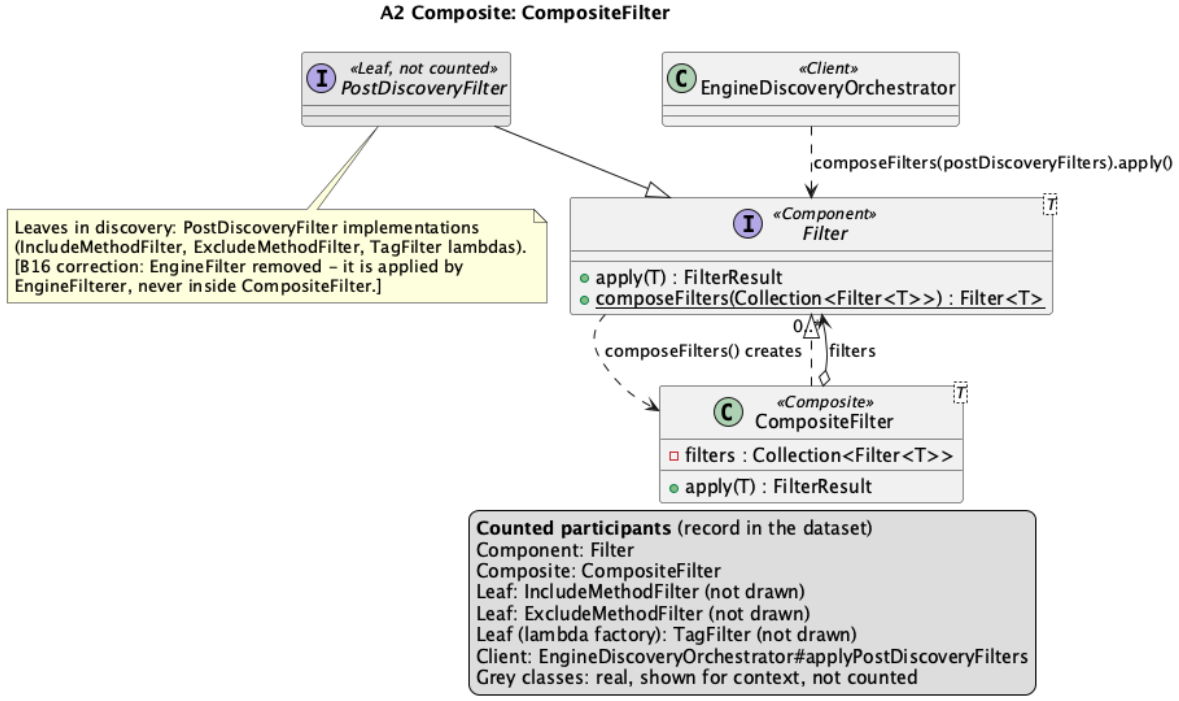


Figure 3: A2 Composed Discovery Filter (Composite), baseline

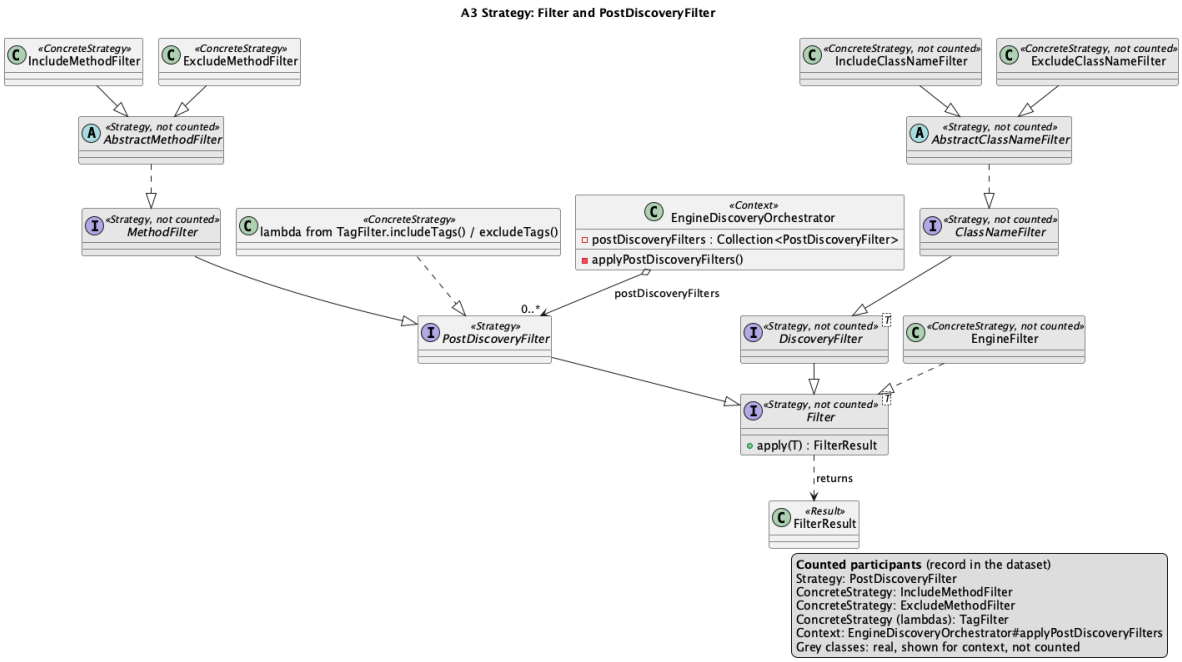


Figure 4: A3 Post-Discovery Filter Policies (Strategy), baseline

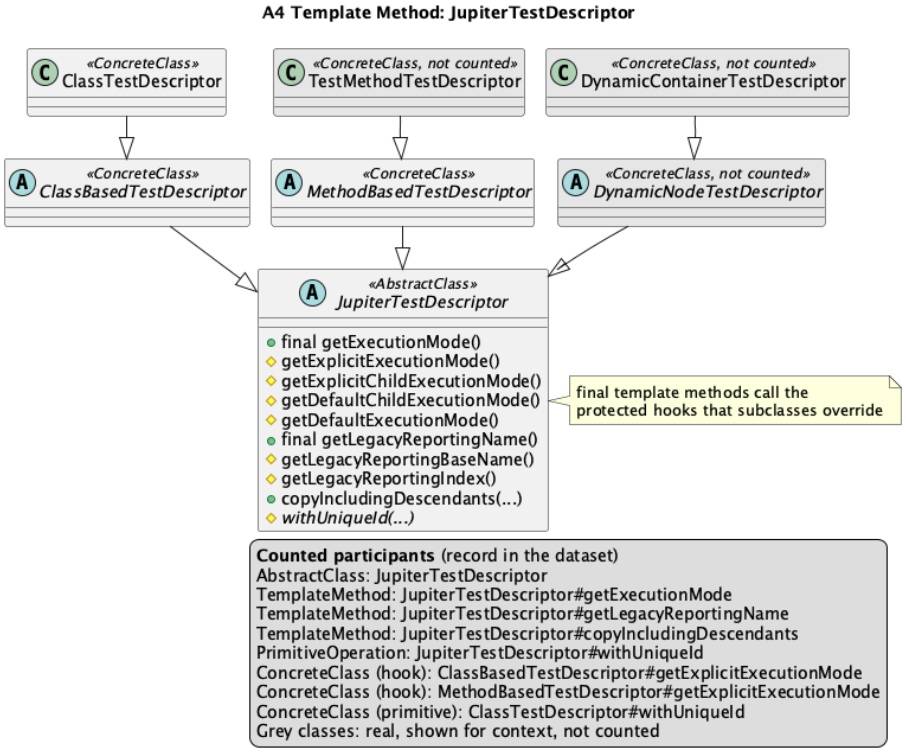


Figure 5: A4 Jupiter Descriptor Skeletons (Template Method), baseline

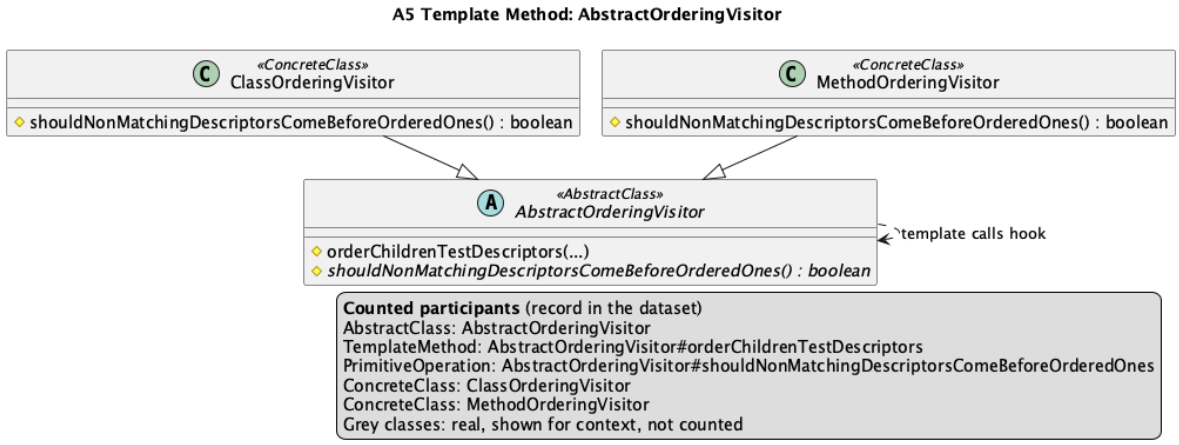


Figure 6: A5 Child Ordering Skeleton (Template Method), baseline

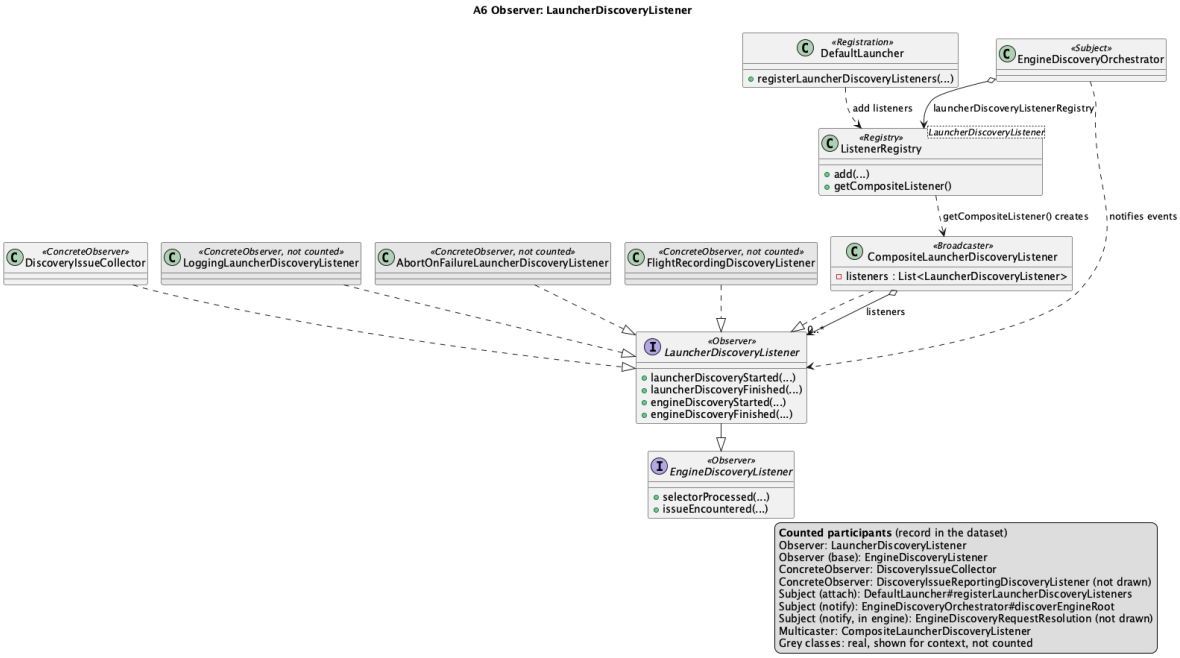


Figure 7: A6 Discovery Event Notification (Observer), baseline

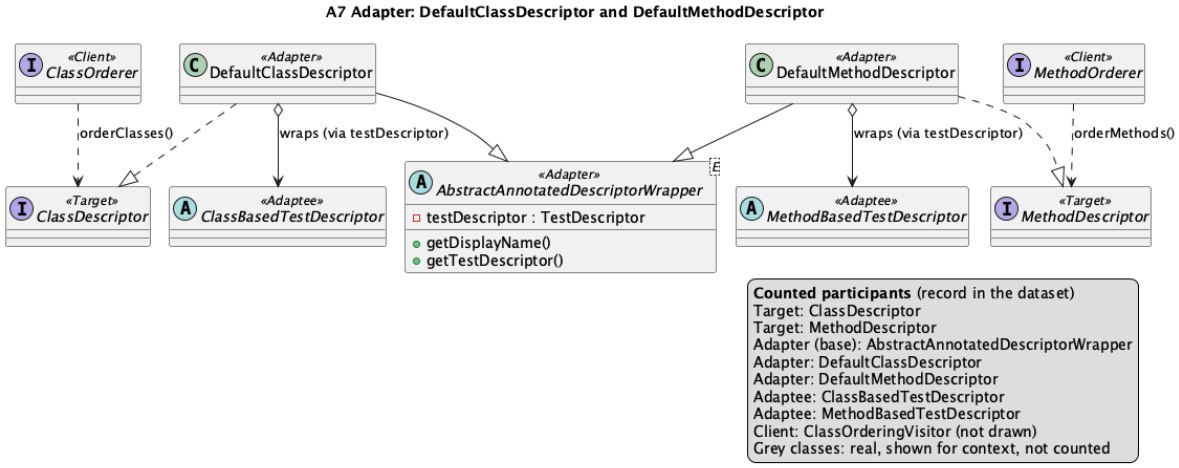


Figure 8: A7 Orderer Descriptor Wrappers (Adapter), baseline

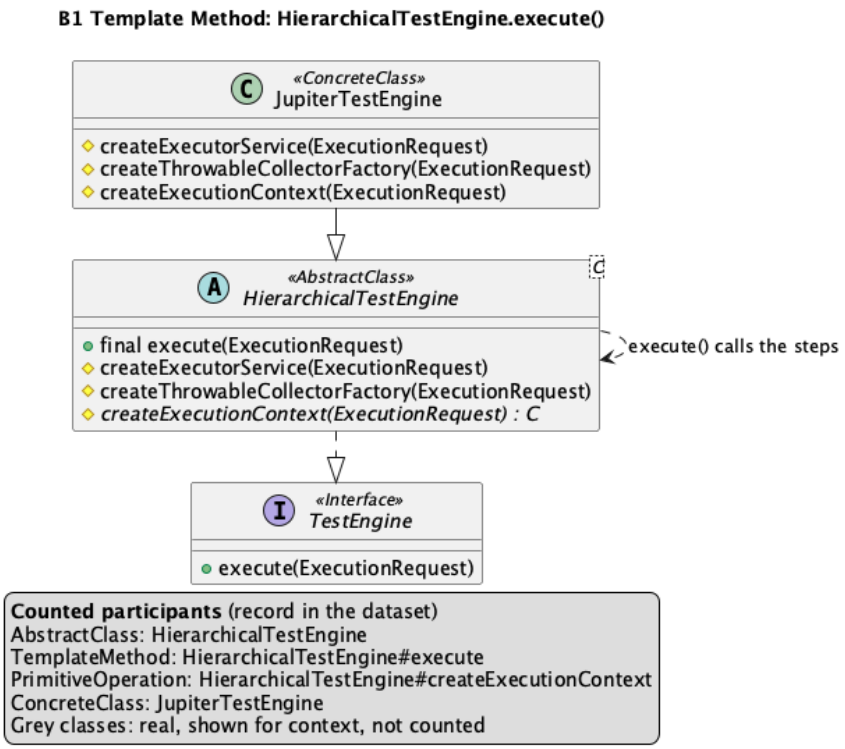


Figure 9: B1 Hierarchical Engine Execution Skeleton (Template Method), baseline

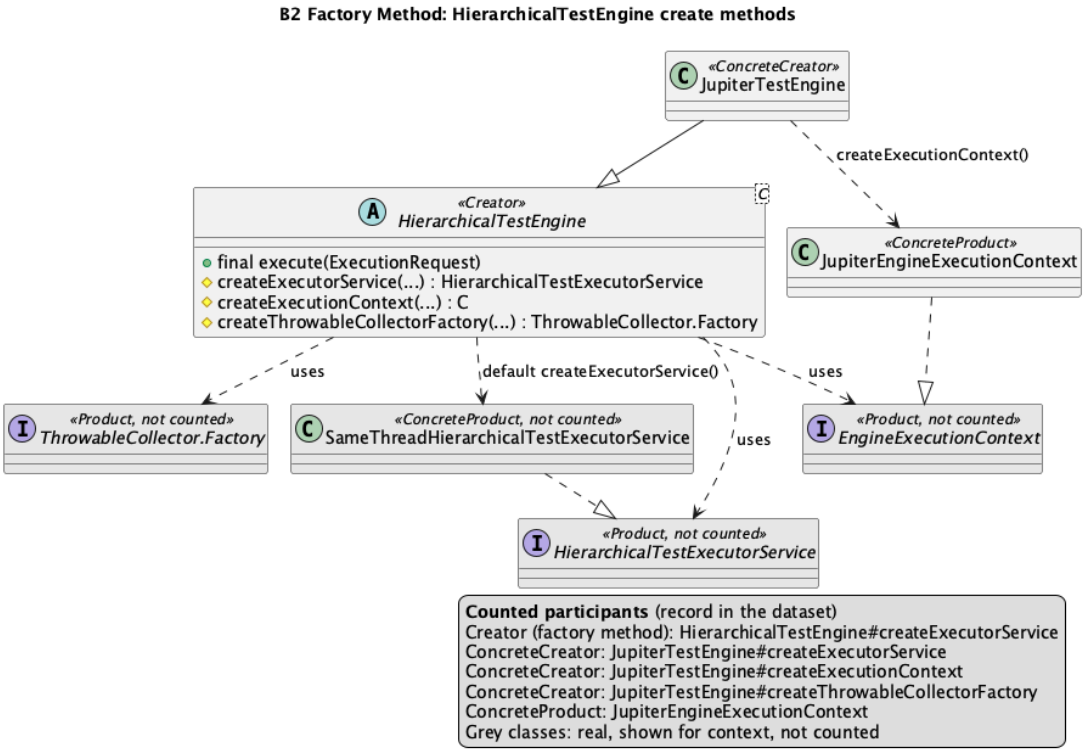


Figure 10: B2 Engine Collaborator Factory Methods (Factory Method), baseline

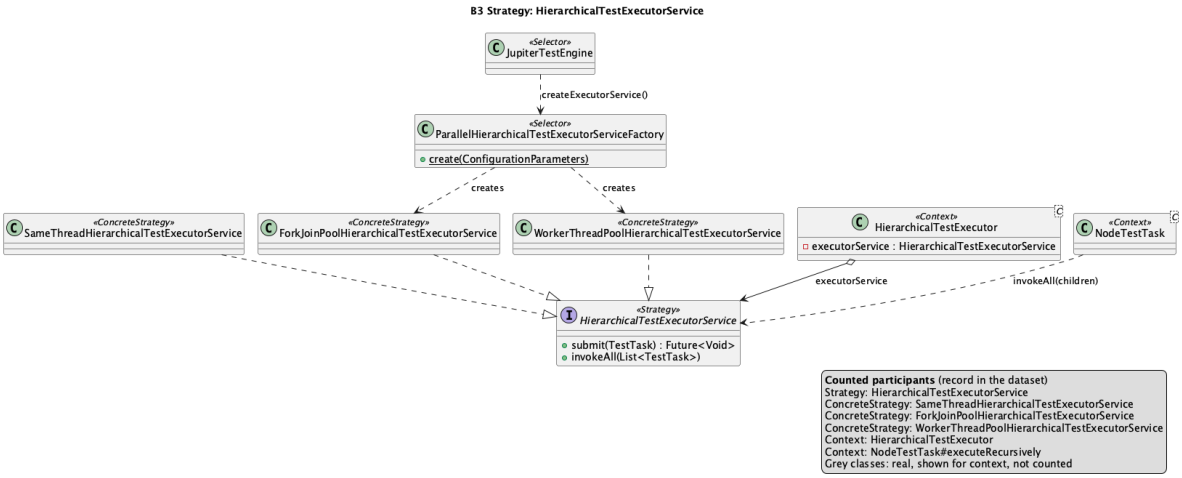


Figure 11: B3 Test Executor Services (Strategy), baseline

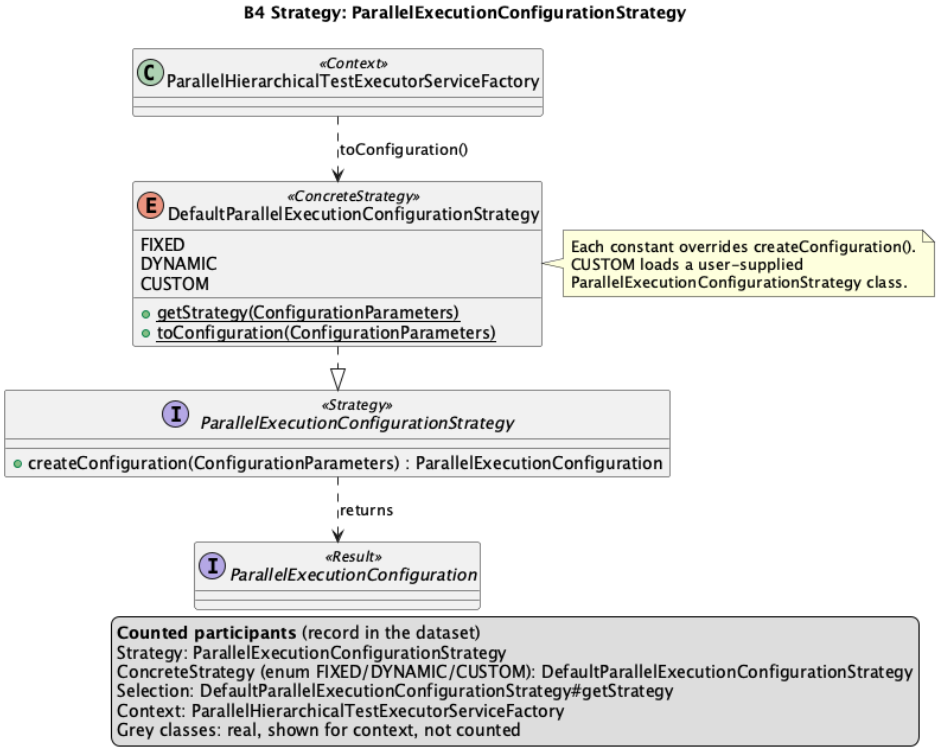


Figure 12: B4 Parallelism Configuration Strategy (Strategy), baseline

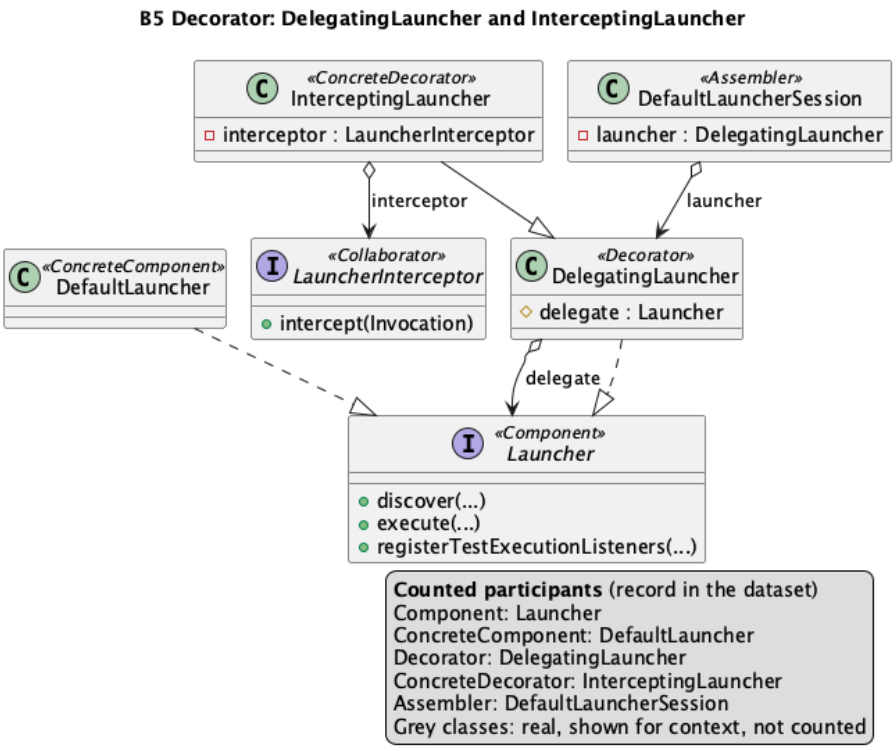


Figure 13: B5 Intercepting Launcher Wrappers (Decorator), baseline

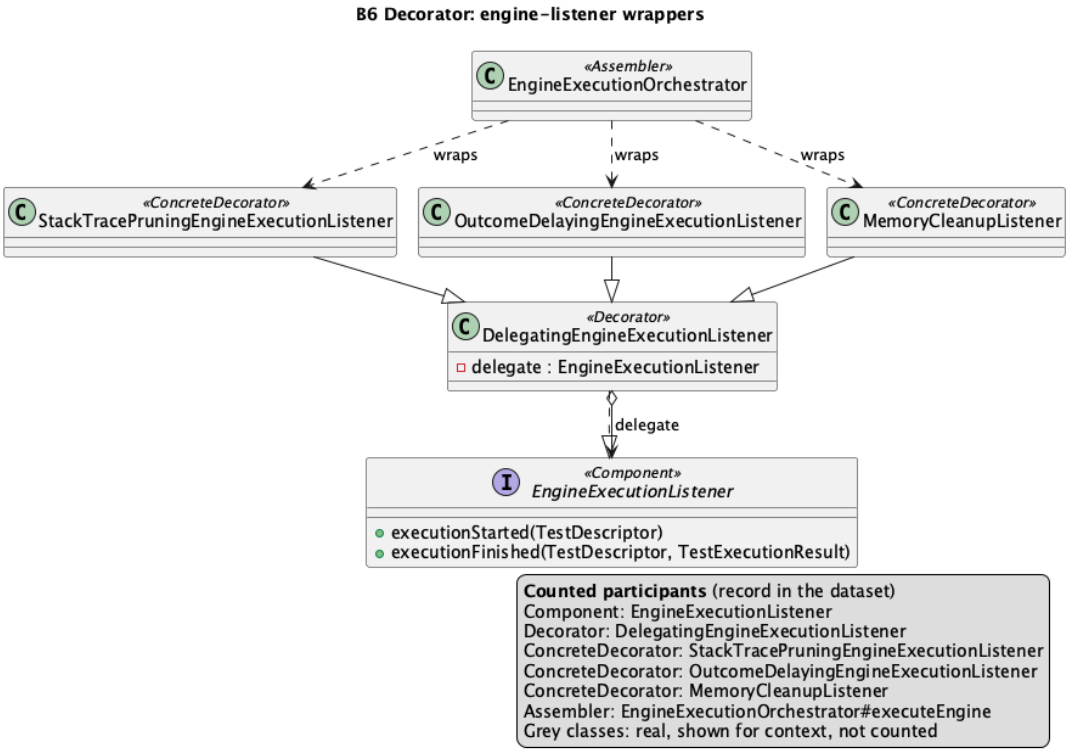


Figure 14: B6 Engine Listener Wrappers (Decorator), baseline

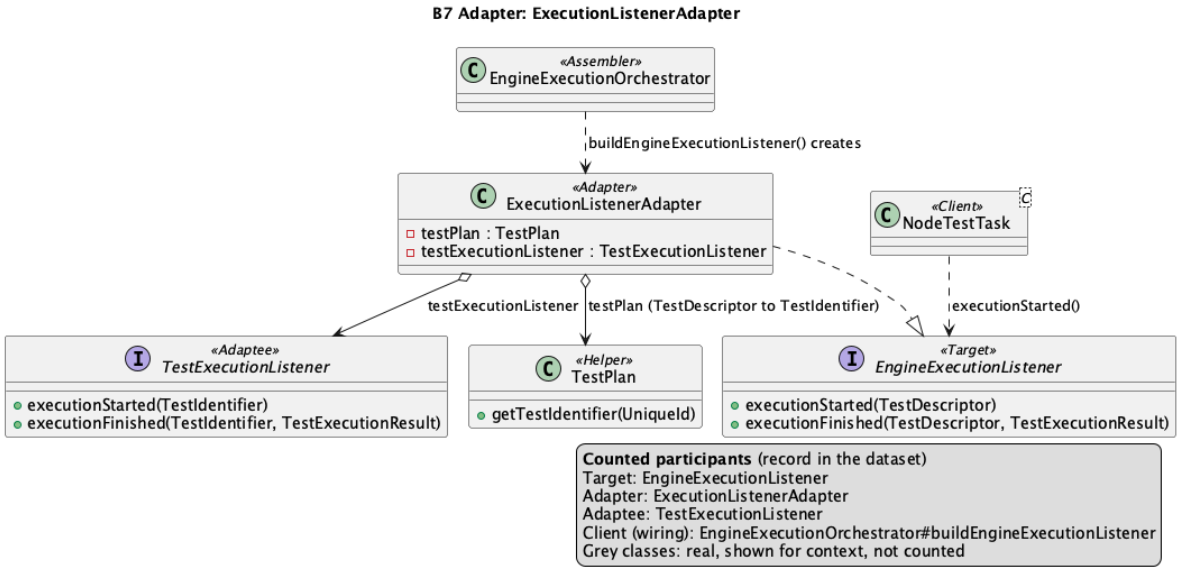


Figure 15: B7 Engine-to-Launcher Listener Adapter (Adapter), baseline

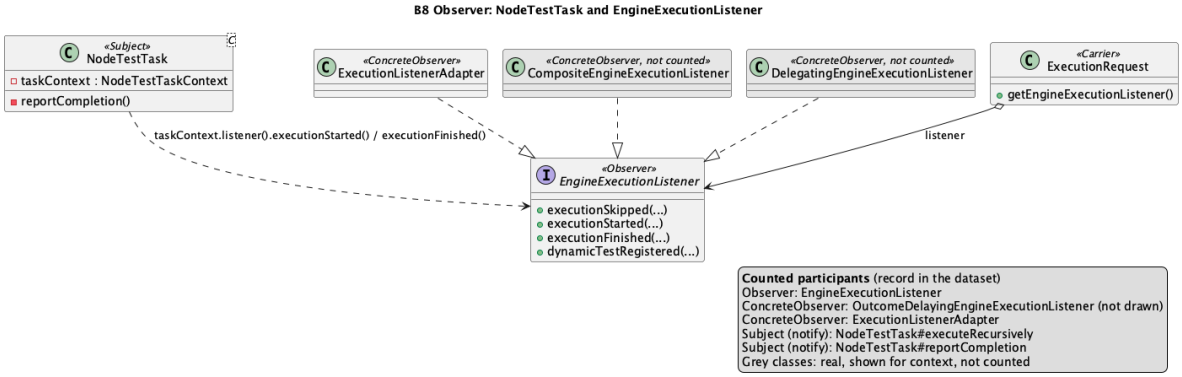


Figure 16: B8 Execution Event Notification (Observer), baseline

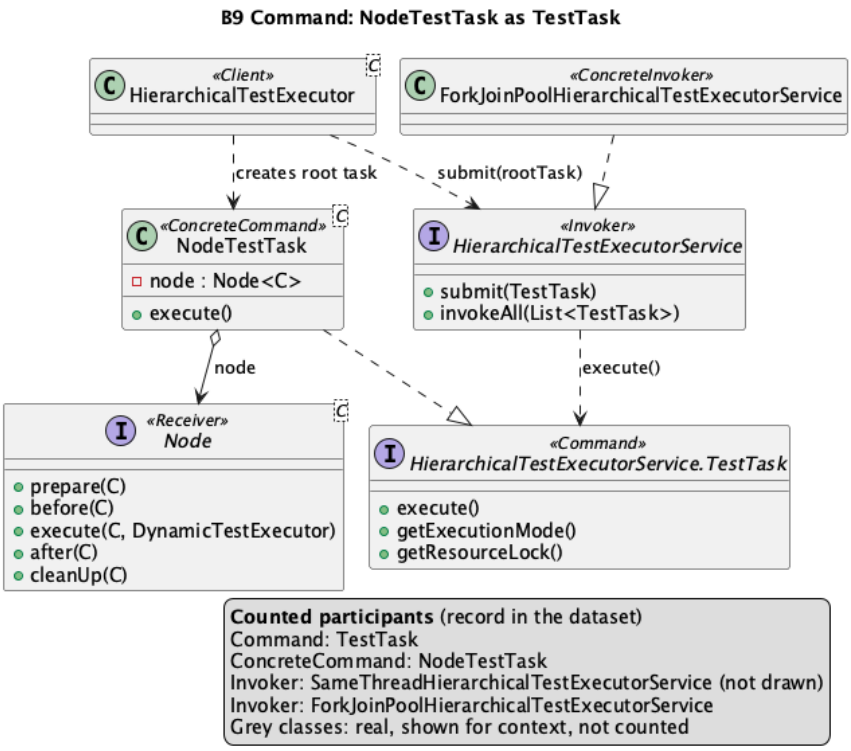


Figure 17: B9 Node Execution Tasks (Command), baseline

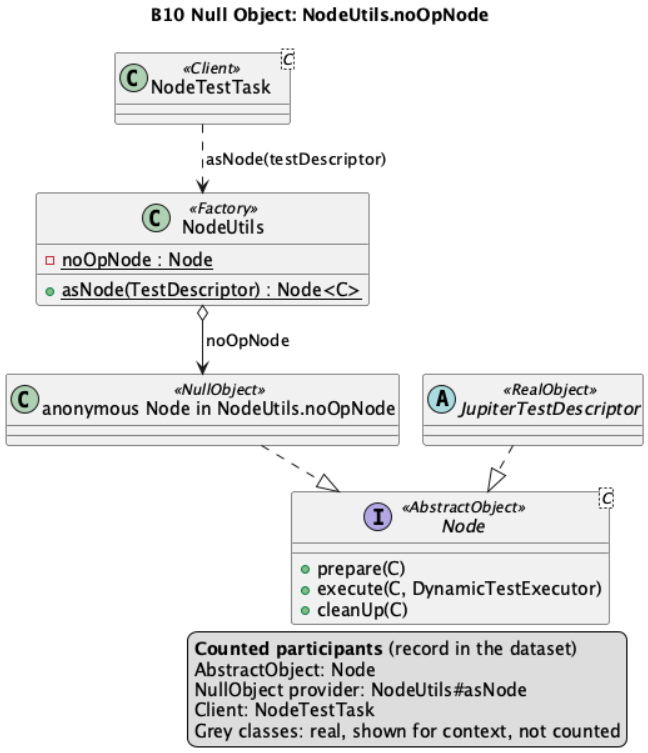


Figure 18: B10 No-op Node (Null Object), baseline

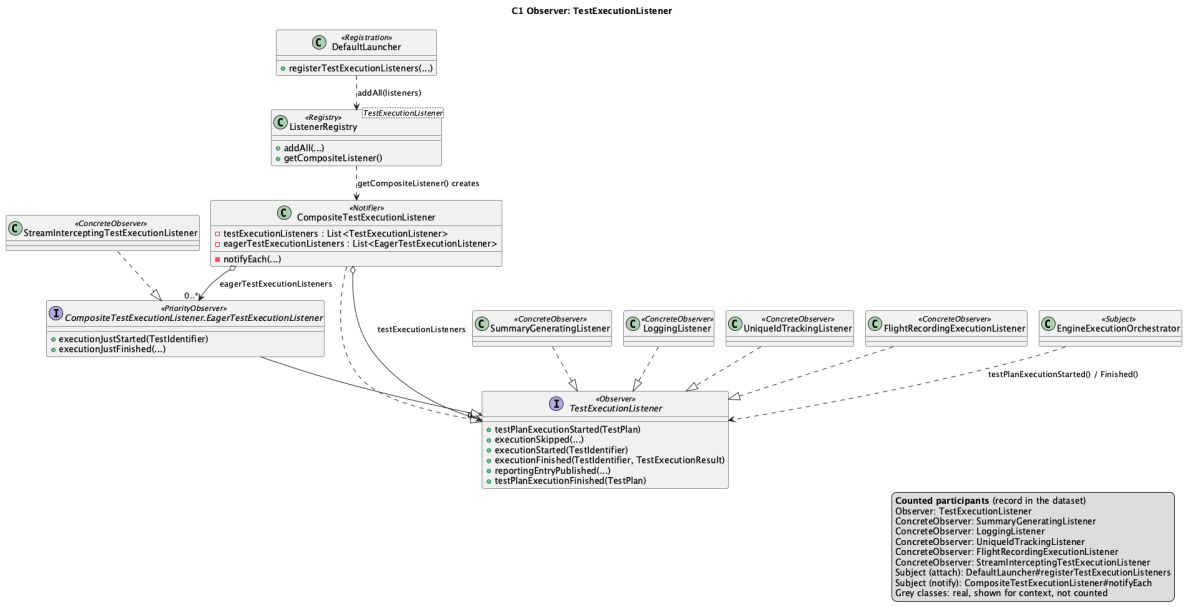


Figure 19: C1 Test Execution Listeners (Observer), baseline

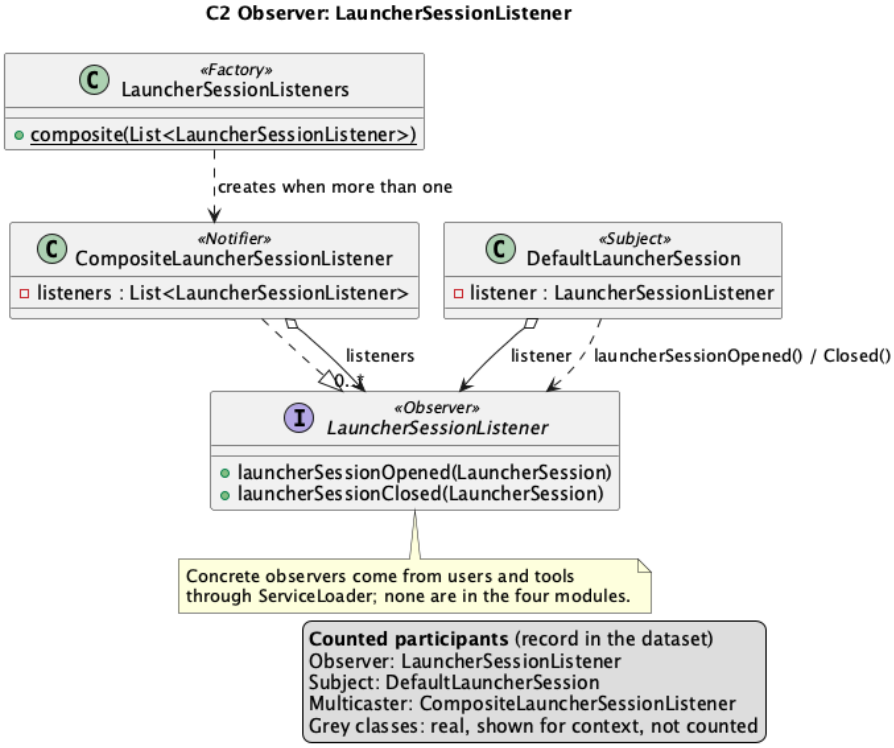


Figure 20: C2 Launcher Session Notification (Observer), baseline

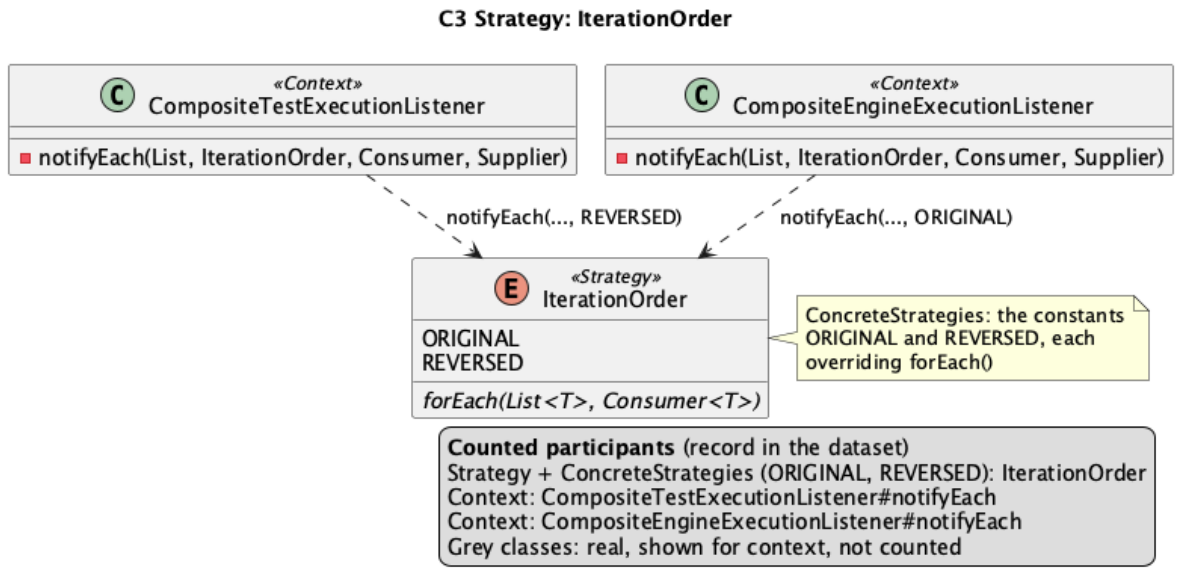


Figure 21: C3 Listener Notification Order (Strategy), baseline

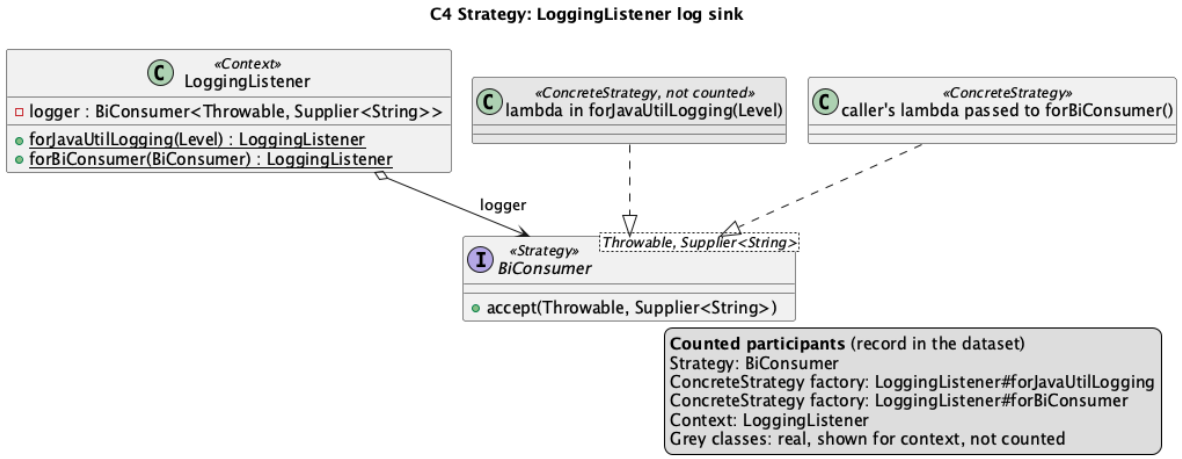


Figure 22: C4 Pluggable Log Sink (Strategy), baseline

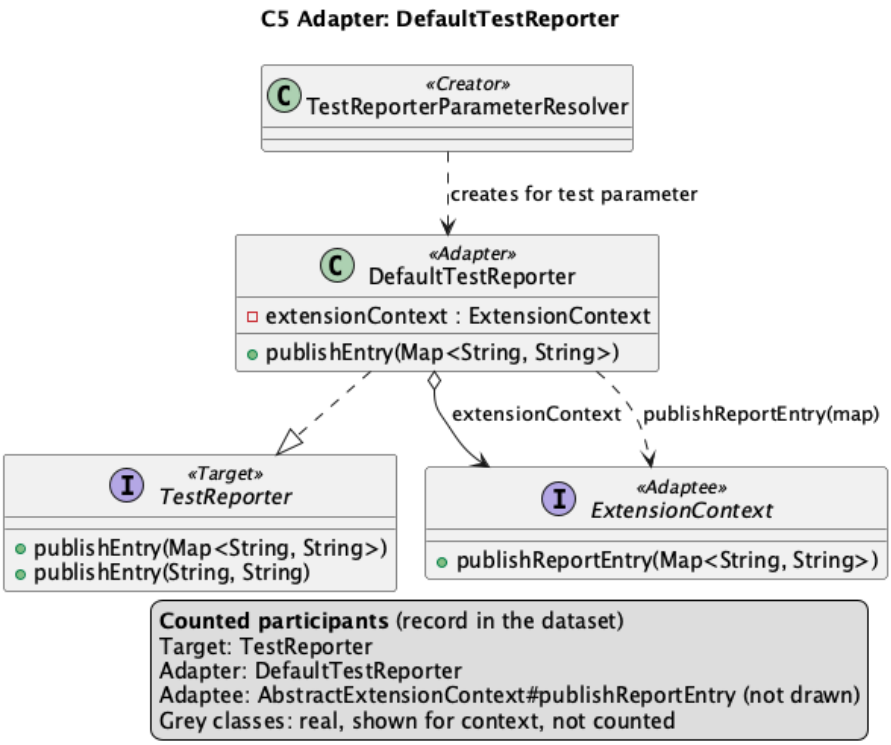


Figure 23: C5 Test Reporter Adapter (Adapter), baseline

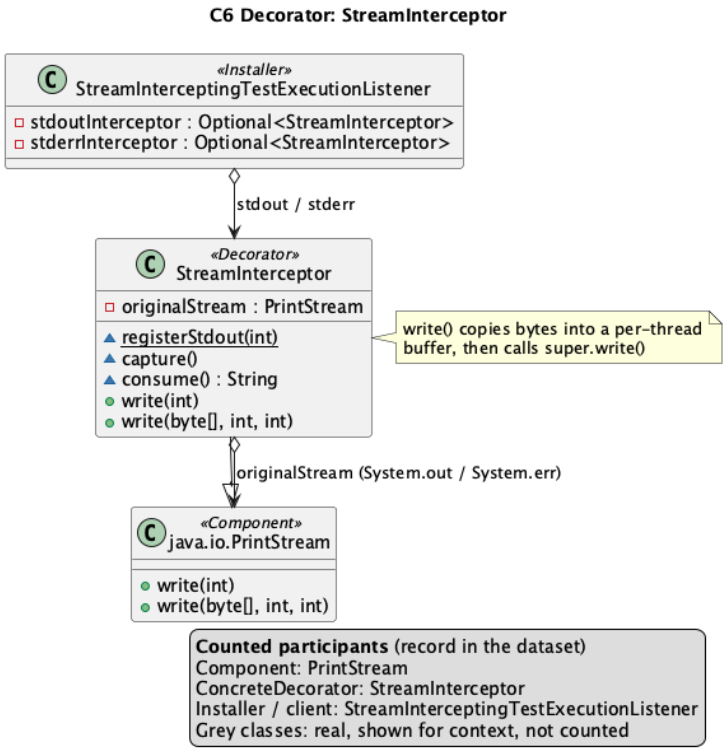


Figure 24: C6 Captured Output Stream (Decorator), baseline

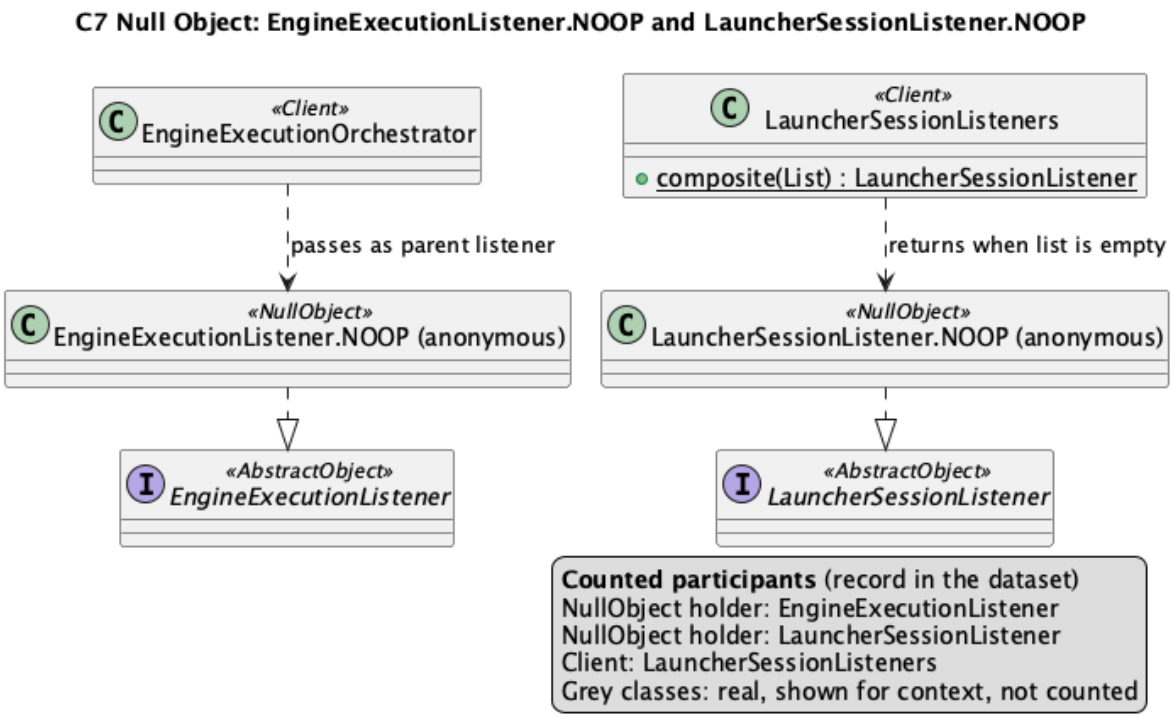


Figure 25: C7 No-op Listeners (Null Object), baseline

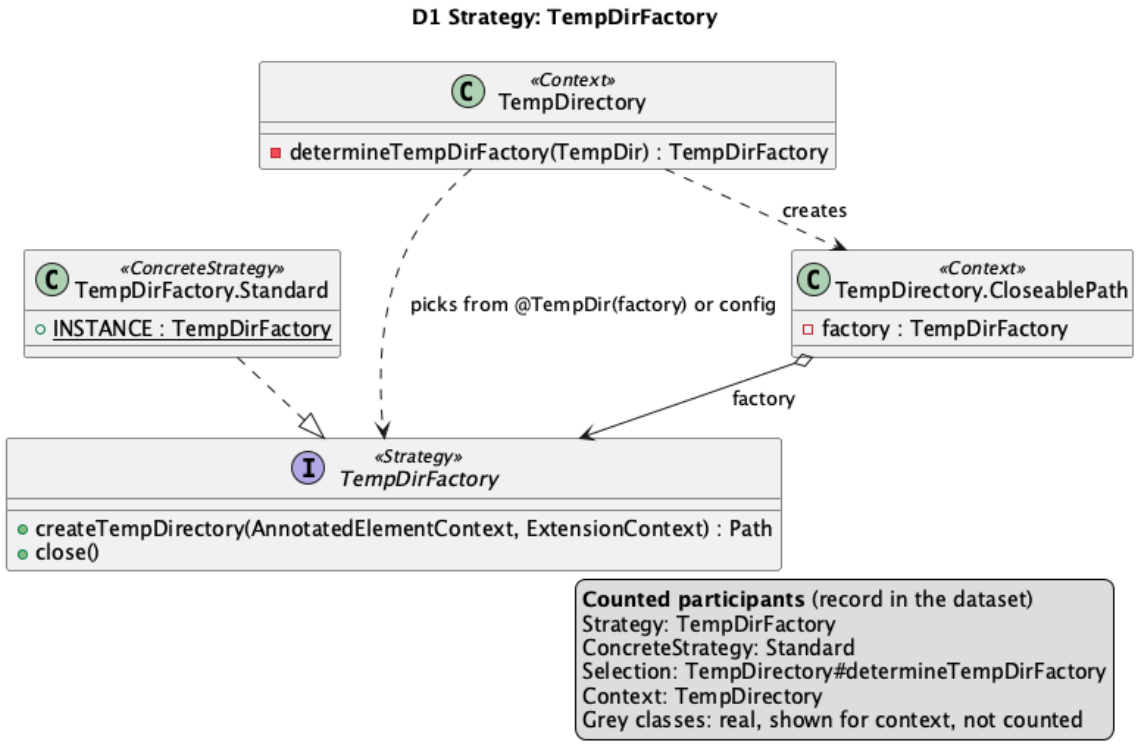


Figure 26: D1 Temp Directory Creation (Strategy), baseline

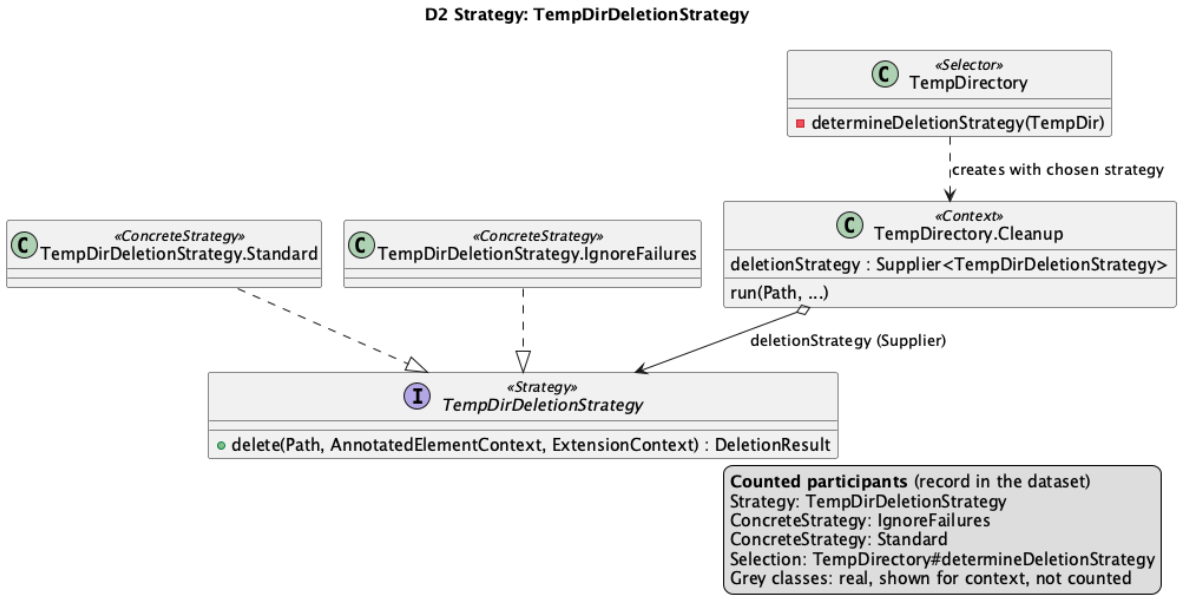


Figure 27: D2 Temp Directory Deletion (Strategy), baseline

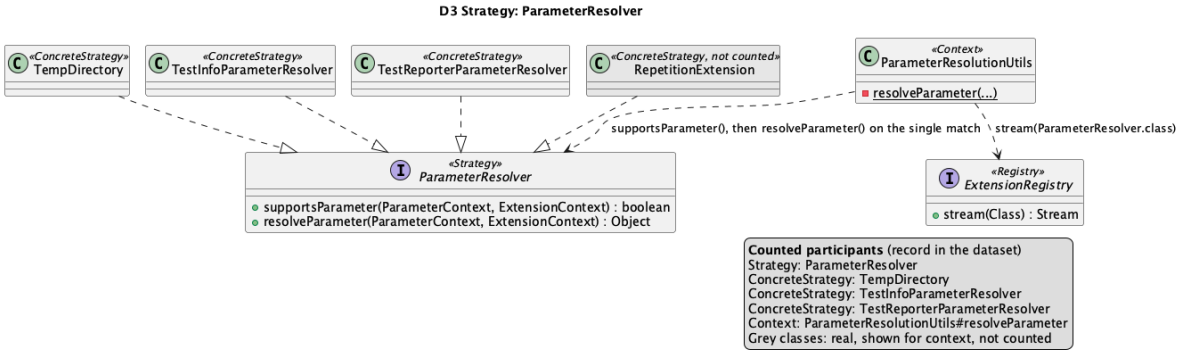


Figure 28: D3 Parameter Resolution (Strategy), baseline

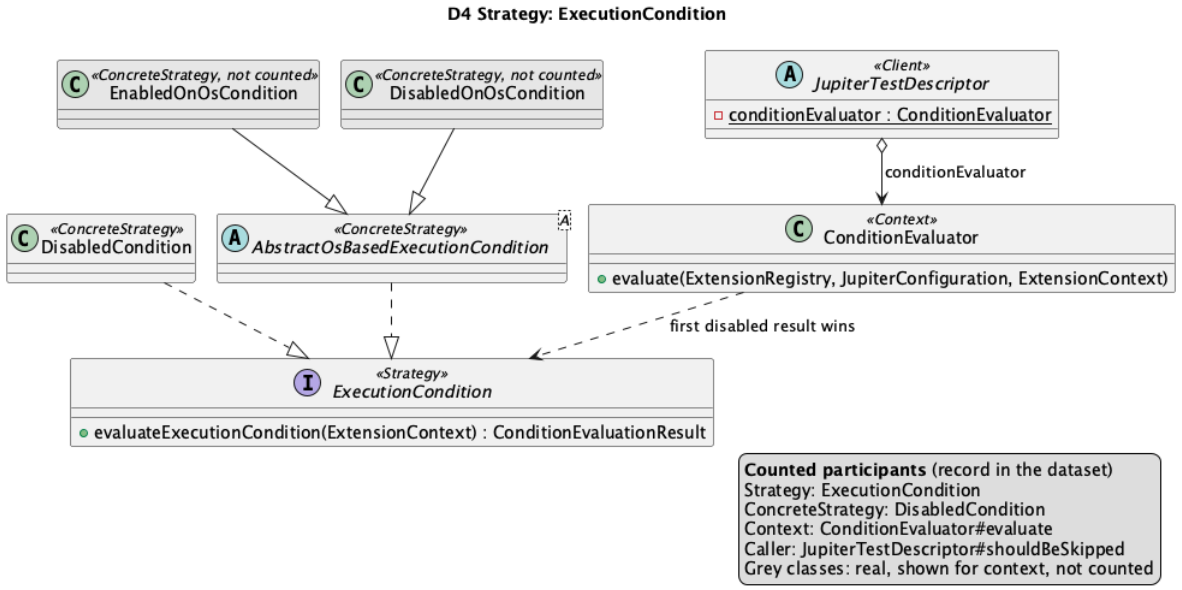


Figure 29: D4 Conditional Execution (Strategy), baseline

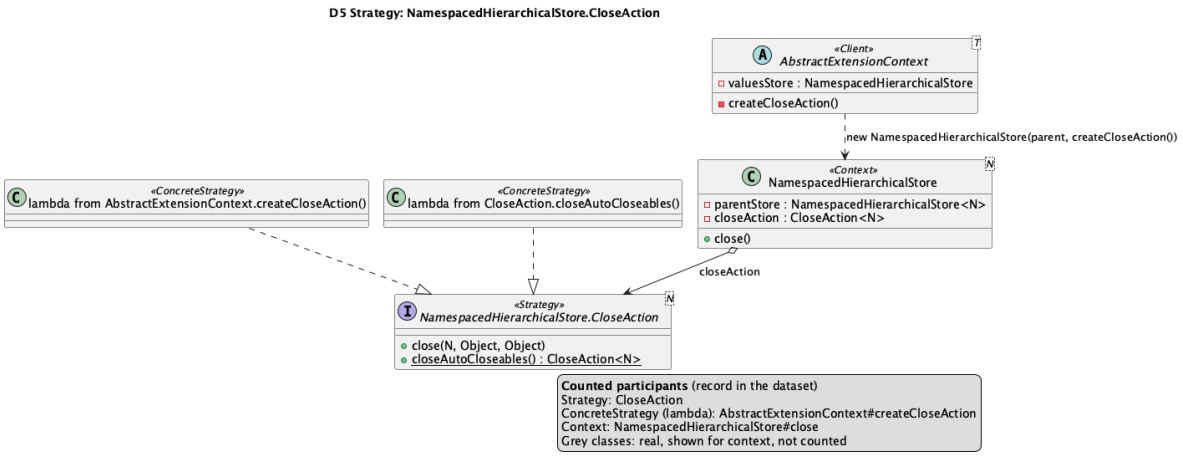


Figure 30: D5 Store Close Action (Strategy), baseline

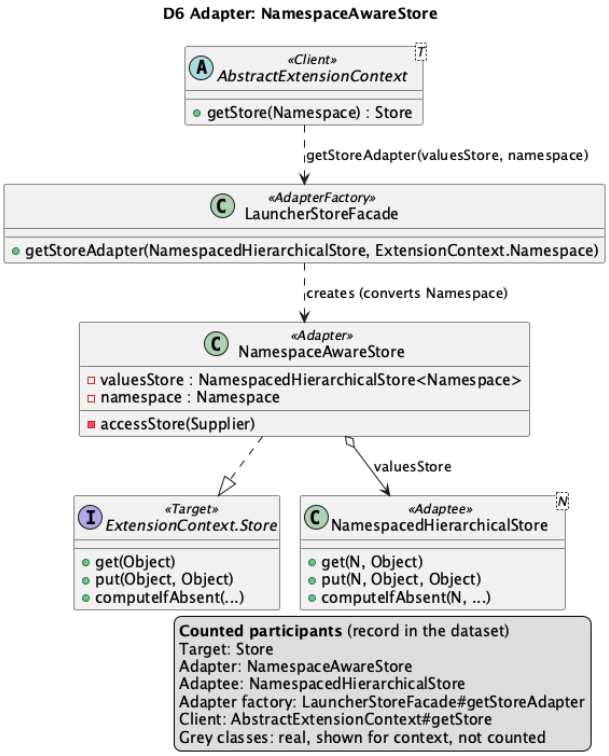


Figure 31: D6 Extension Store Adapter (Adapter), baseline

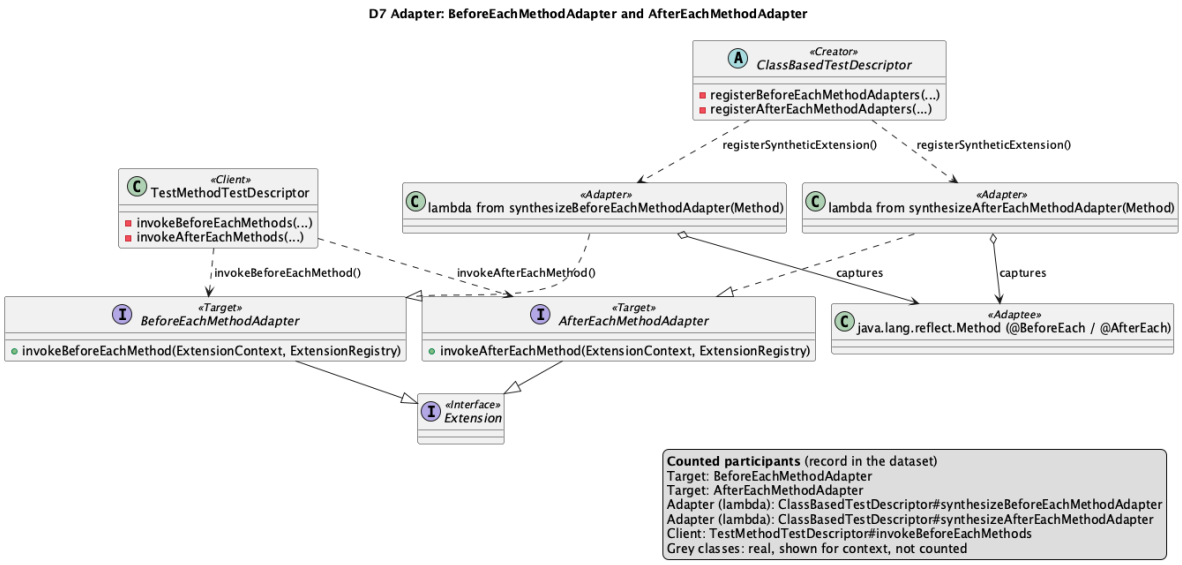


Figure 32: D7 Lifecycle Method Adapters (Adapter), baseline

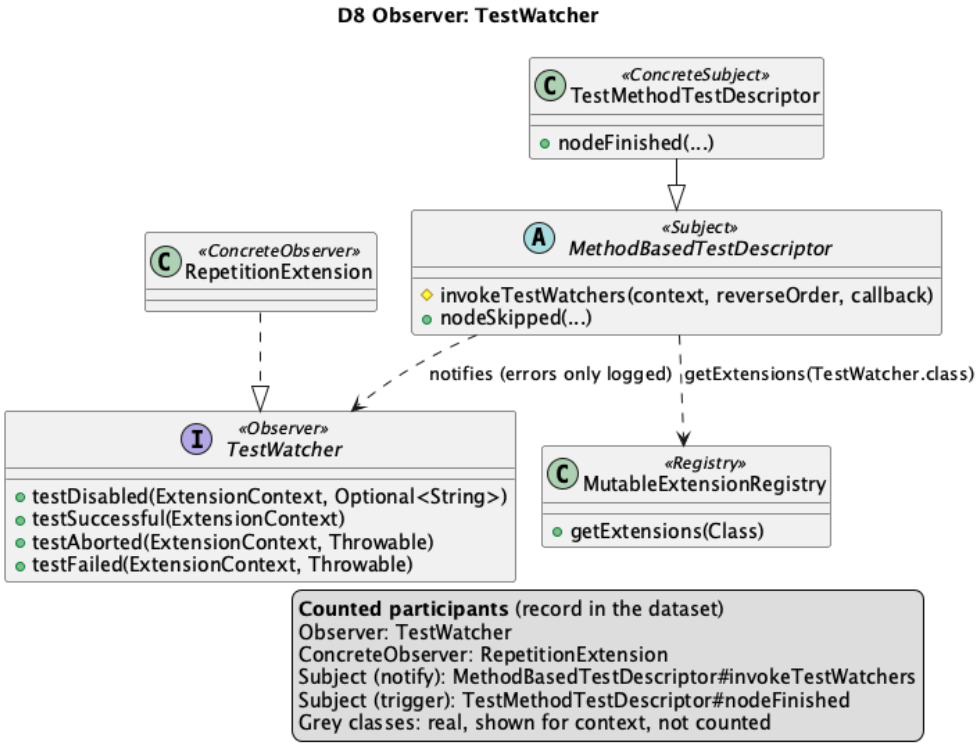


Figure 33: D8 Test Outcome Watchers (Observer), baseline

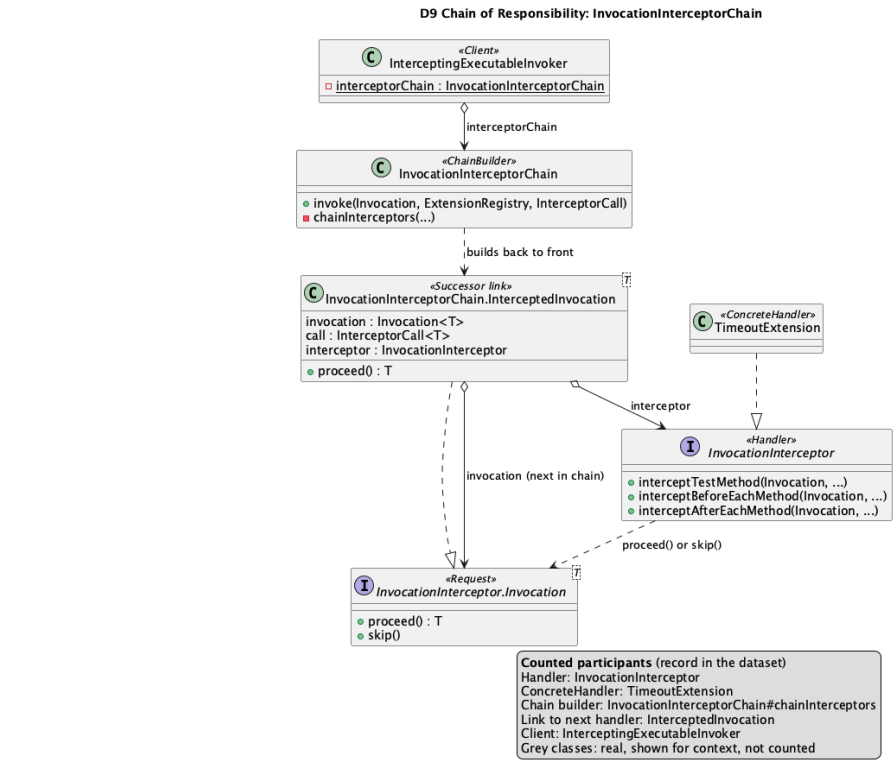


Figure 34: D9 Invocation Interceptor Chain (Chain of Responsibility), baseline

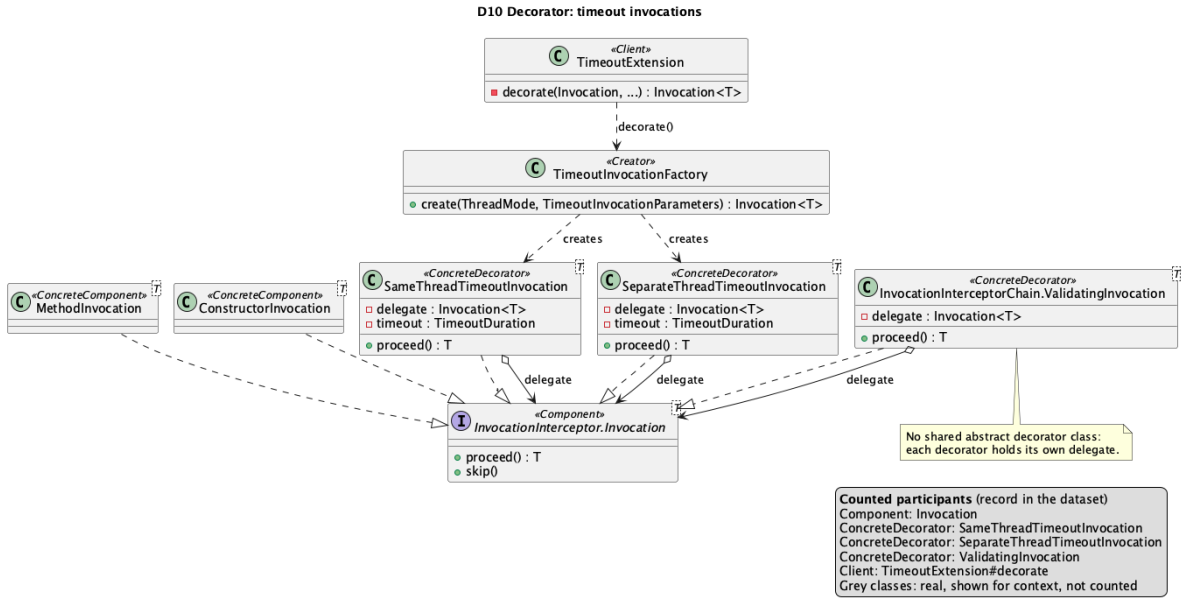


Figure 35: D10 Timeout Invocation Wrappers (Decorator), baseline

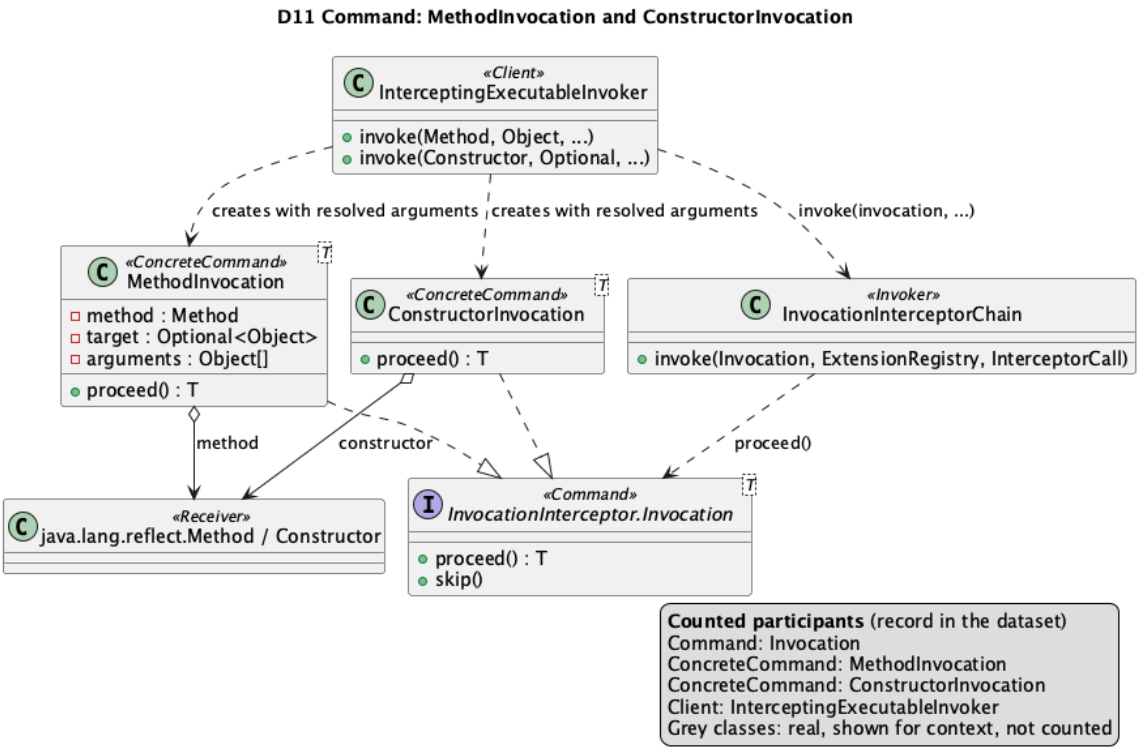


Figure 36: D11 Packaged Reflective Calls (Command), baseline

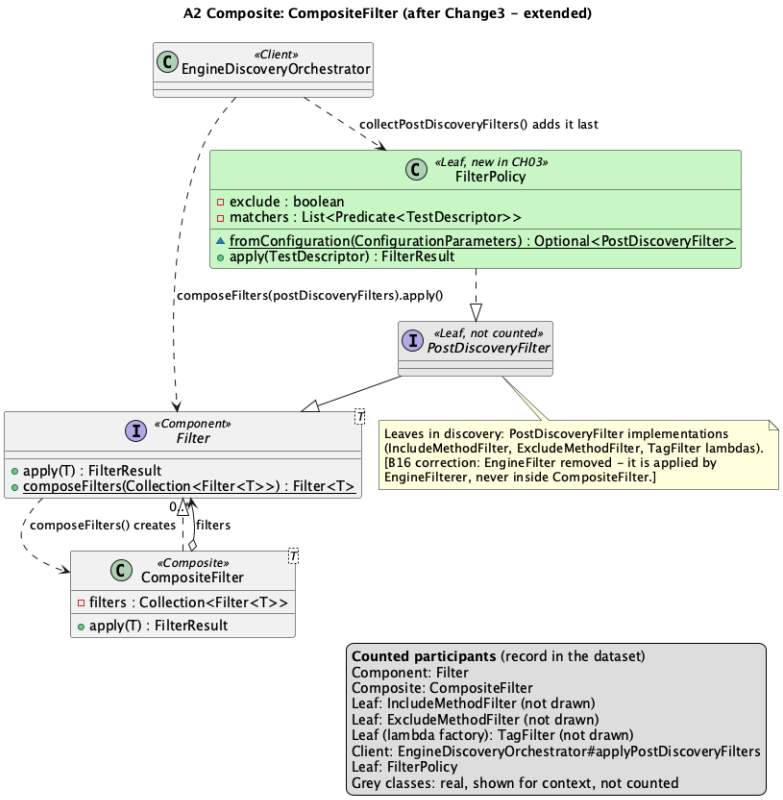


Figure 37: A2 after Change3 (extended)

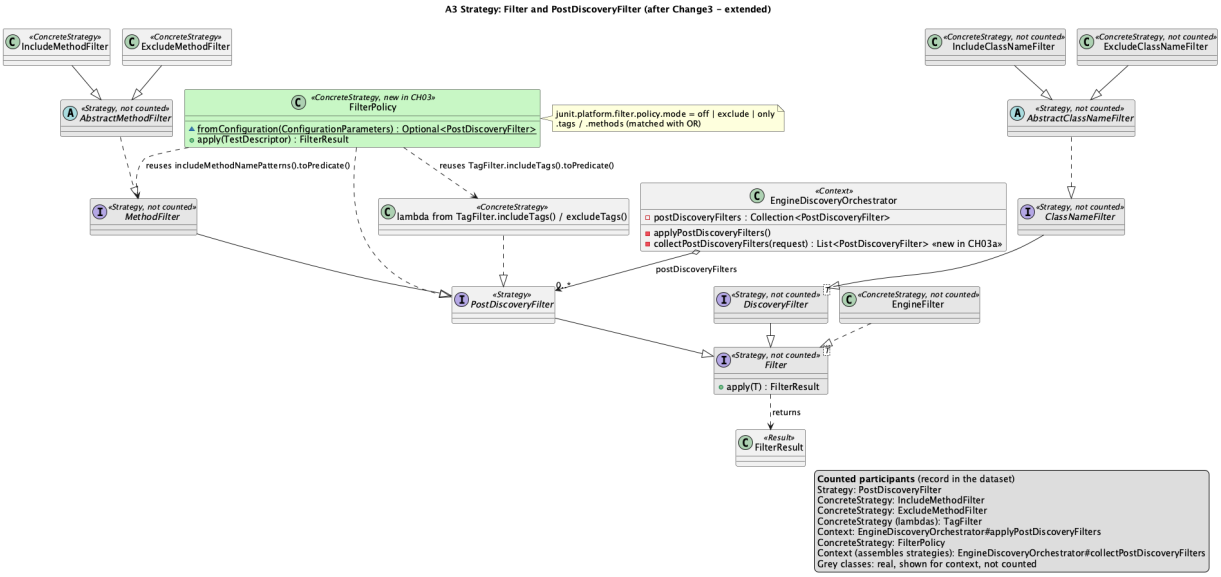


Figure 38: A3 after Change3 (extended)

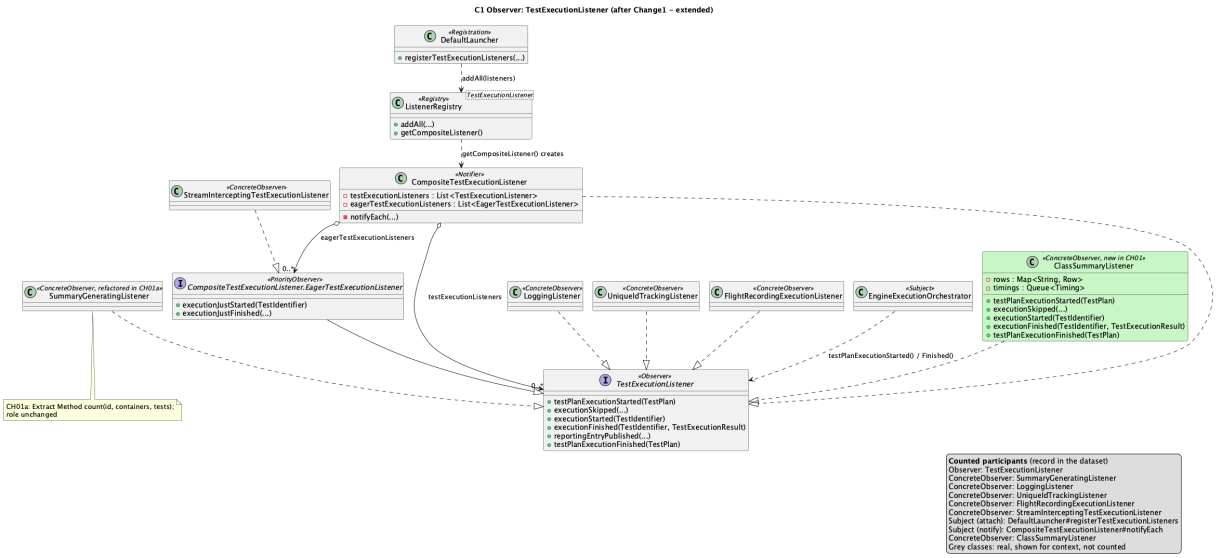


Figure 39: C1 after Change1 (extended)

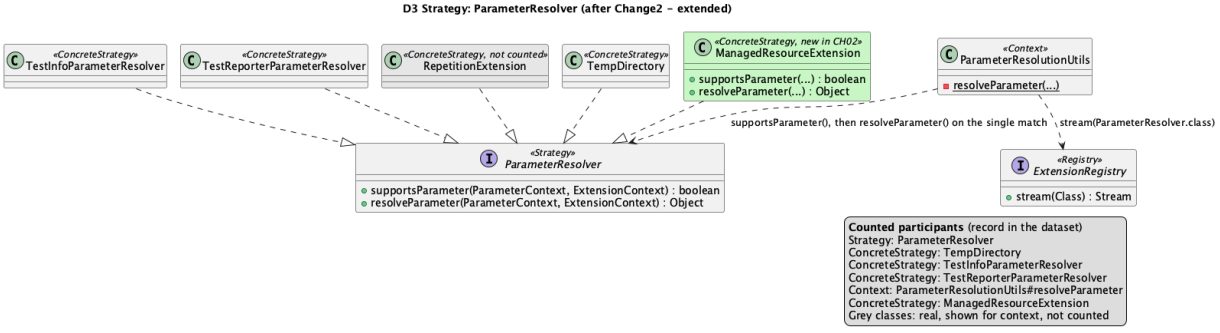


Figure 40: D3 after Change2 (extended)

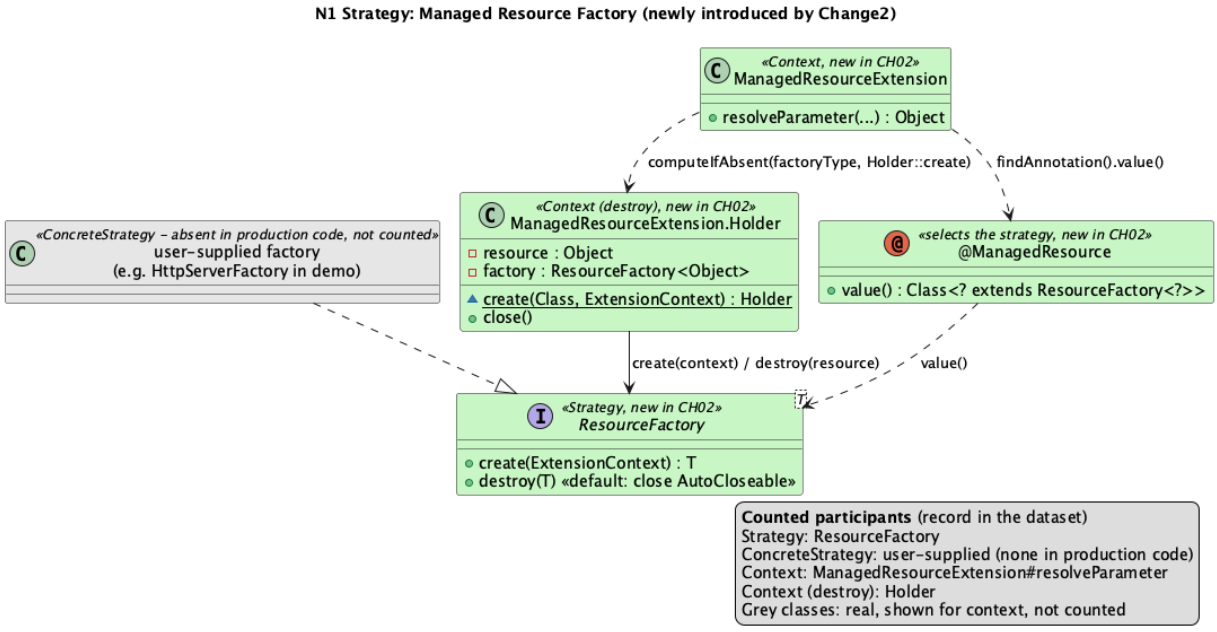


Figure 41: N1 Managed Resource Factory (Strategy, newly introduced by Change2)

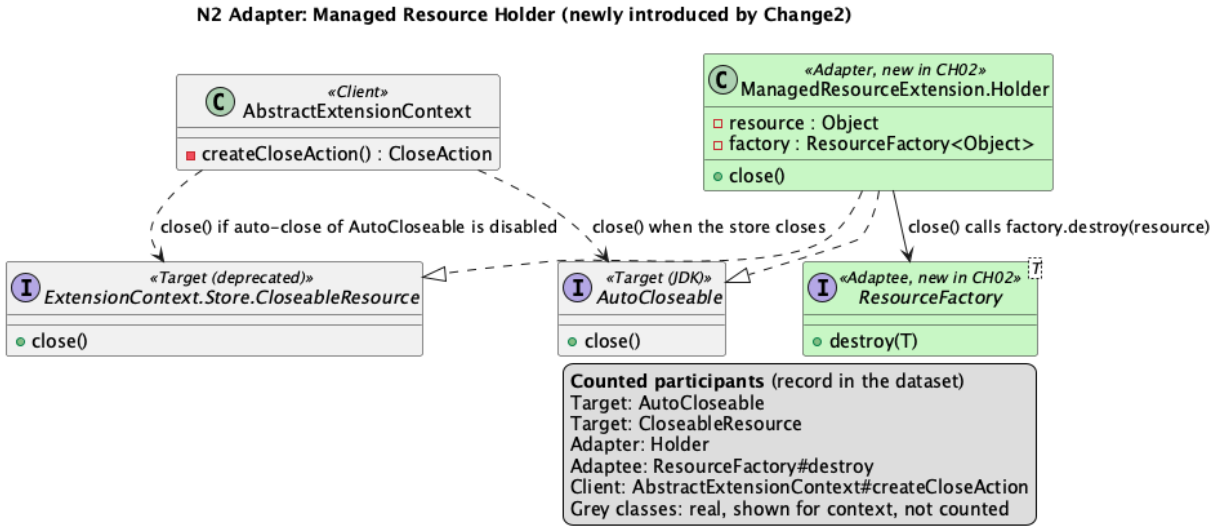


Figure 42: N2 Managed Resource Holder (Adapter, newly introduced by Change2)